

Introduction to Visual Studio and Visual Programming

Chapter 2 of Visual C# How to Program, 6/e

OBJECTIVES

In this chapter you'll:

- Learn the basics of the Visual Studio Community 2015 Integrated Development Environment (IDE) for writing, running and debugging your apps.
- Create a new project using Visual Studio's **Windows Forms Application** template.
- Be introduced to Windows **Forms** and the controls you'll use to build graphical user interfaces.
- Use key commands contained in the IDE's menus and toolbars.
- Understand the purpose of the various windows in the Visual Studio.
- Use Visual Studio's help features.
- Use visual app development to conveniently create, compile and execute a simple Visual C# app that displays text and an image.

2.1 Introduction

2.2 Overview of the Visual Studio Community 2015 IDE

2.2.1 Introduction to Visual Studio Community 2015

2.2.2 Visual Studio Themes

2.2.3 Links on the Start Page

2.2.4 Creating a New Project

2.2.5 **New Project** Dialog and Project Templates

2.2.6 Forms and Controls

2.3 Menu Bar and Toolbar

2.4 Navigating the Visual Studio IDE

2.4.1 **Solution Explorer**

2.4.2 **Toolbox**

2.4.3 **Properties** Window

2.5 Help Menu and Context-Sensitive Help

2.6 Visual Programming: Creating a Simple App that Displays Text and an Image

2.7 Wrap-Up

2.8 Web Resources

2.1 Introduction

- ▶ **Visual Studio** is Microsoft's Integrated Development Environment (IDE) for creating, running and debugging apps (also called applications) written in C# and various other .NET programming languages.
- ▶ In this chapter, we overview the Visual Studio Community 2015 IDE, then show how to create a simple Visual C# app by dragging and dropping predefined building blocks into place—a technique known as **visual app development**.

2.2 Overview of the Visual Studio Community 2015 IDE



- ▶ This book's examples, screen captures and discussions are based on the free **Visual Studio Community 2015** running on Windows 10.
- ▶ The examples will work on full versions of Visual Studio as well—though some options, menus and instructions might differ.
- ▶ From this point forward, we'll refer to Visual Studio Community 2015 simply as “Visual Studio” or “the IDE.” We assume that you have some familiarity with Windows.

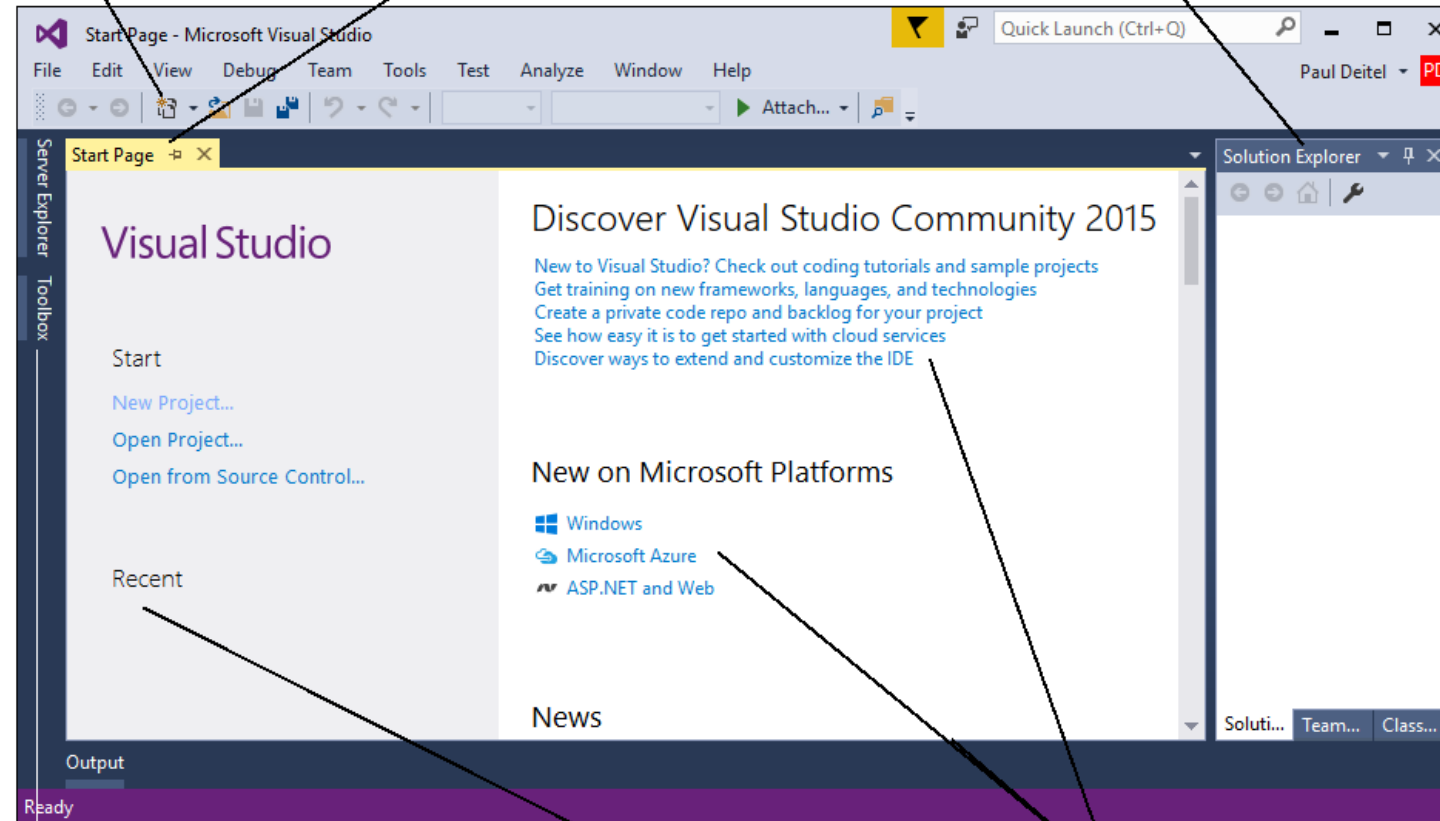
2.2.1 Introduction to Visual Studio Community 2015

- ▶ When you open Visual Studio it initially displays the **Start Page** (Fig. 2.1).
- ▶ Depending on your version of Visual Studio, your Start Page may look different.
- ▶ The Start Page contains a list of links to Visual Studio resources and web-based resources.
- ▶ At any time, you can return to the **Start Page** by selecting **View > Start Page**.

New Project button

Start Page tab

Solution Explorer (no projects open)



Collapsed **Toolbox** window

Recent projects will be listed here

Start Page links

Fig. 2.1 | **Start Page** in Visual Studio Community 2015.

2.2.2 Visual Studio Themes

- ▶ Visual Studio supports three themes that specify the IDE's color scheme:
 - a *dark theme* (with dark window backgrounds and light text)
 - a *light theme* (with light window backgrounds and dark text) and
 - a *blue theme* (with light window backgrounds and dark text).
- ▶ We use the blue theme throughout this book.
- ▶ The Before You Begin section after the Preface explains how to set this option.

2.2.3 Links on the Start Page

- ▶ The Start Page links are organized into two columns.
 - The left column's **Start** section contains options for building new apps or working on existing ones.
 - The left column's **Recent** section contains links to projects you've recently created or modified.

2.2.3 Links on the Start Page

- ▶ The Start Page's right column contains links to various online documentation and resources to help you get started with Visual Studio and learn about Microsoft programming technologies.
- ▶ To access more extensive information on Visual Studio, you can browse the **MSDN (Microsoft Developer Network)** Library at
 - <https://msdn.microsoft.com/library/dd831853>
- ▶ You also can browse the web from the IDE by selecting **View > Other Windows > Web Browser**.
- ▶ To request a web page, type its URL into the location bar (Fig. 2.2) and press *Enter*

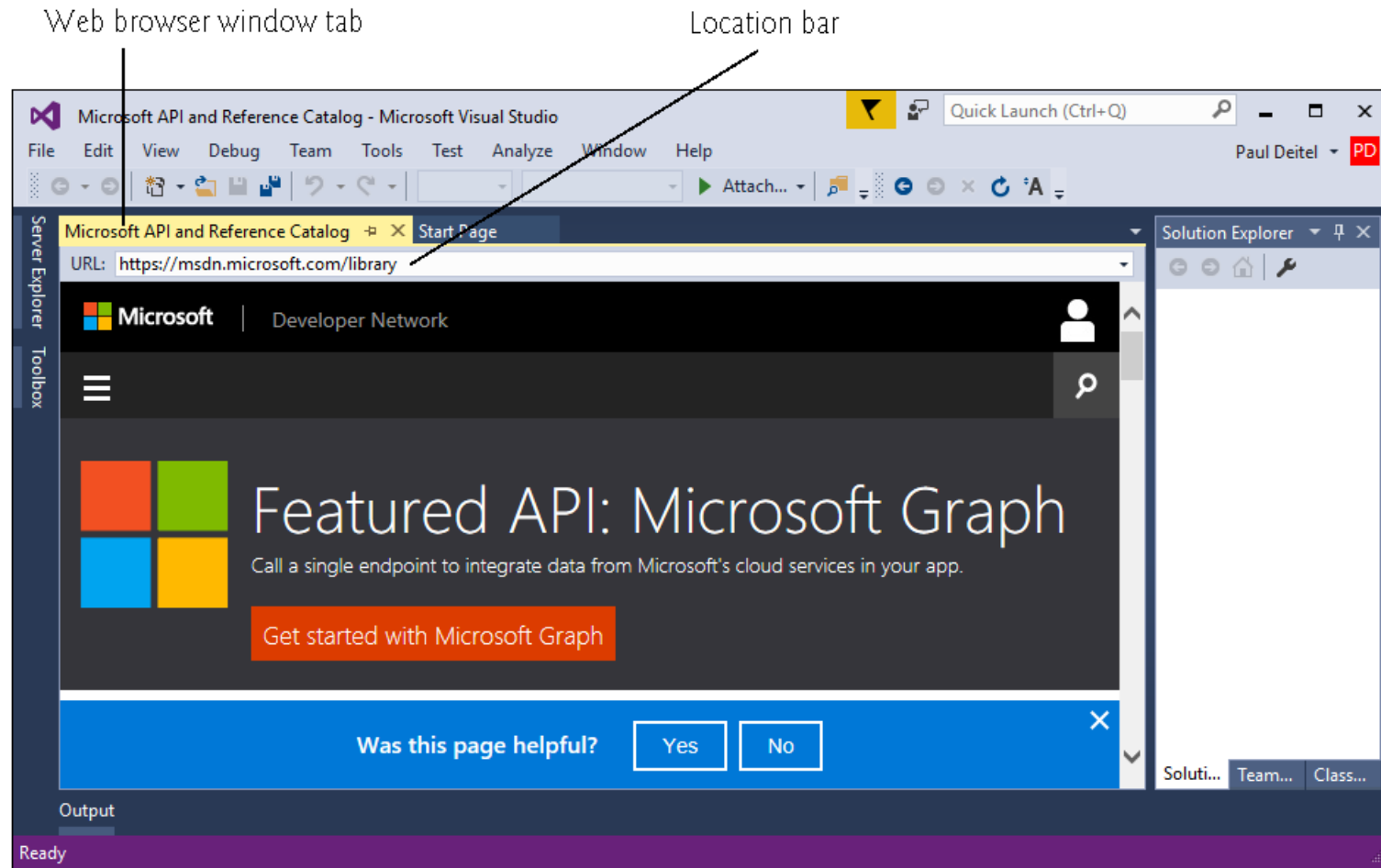


Fig. 2.2 | MSDN Library web page in Visual Studio.

2.2.4 Creating a New Project

- ▶ A **project** is a group of related files, such as the Visual C# code and any images that might make up an app.
- ▶ Visual Studio organizes apps into projects and **solutions**, which contain one or more projects.
 - Multiple-project solutions are used to create large-scale apps.
- ▶ You select **File > New > Project...** to create a new project or **File > Open > Project/Solution...** to open an existing one.
- ▶ You also can click the corresponding links in the **Start Page's Start** section.



2.2.5 New Project Dialog and Project Templates

- ▶ For the discussions in the next several sections, we'll create a new project. Select **File > New > Project...** to display the **New Project dialog** (Fig. 2.3).
- ▶ **Dialogs** are windows that facilitate user–computer communication.

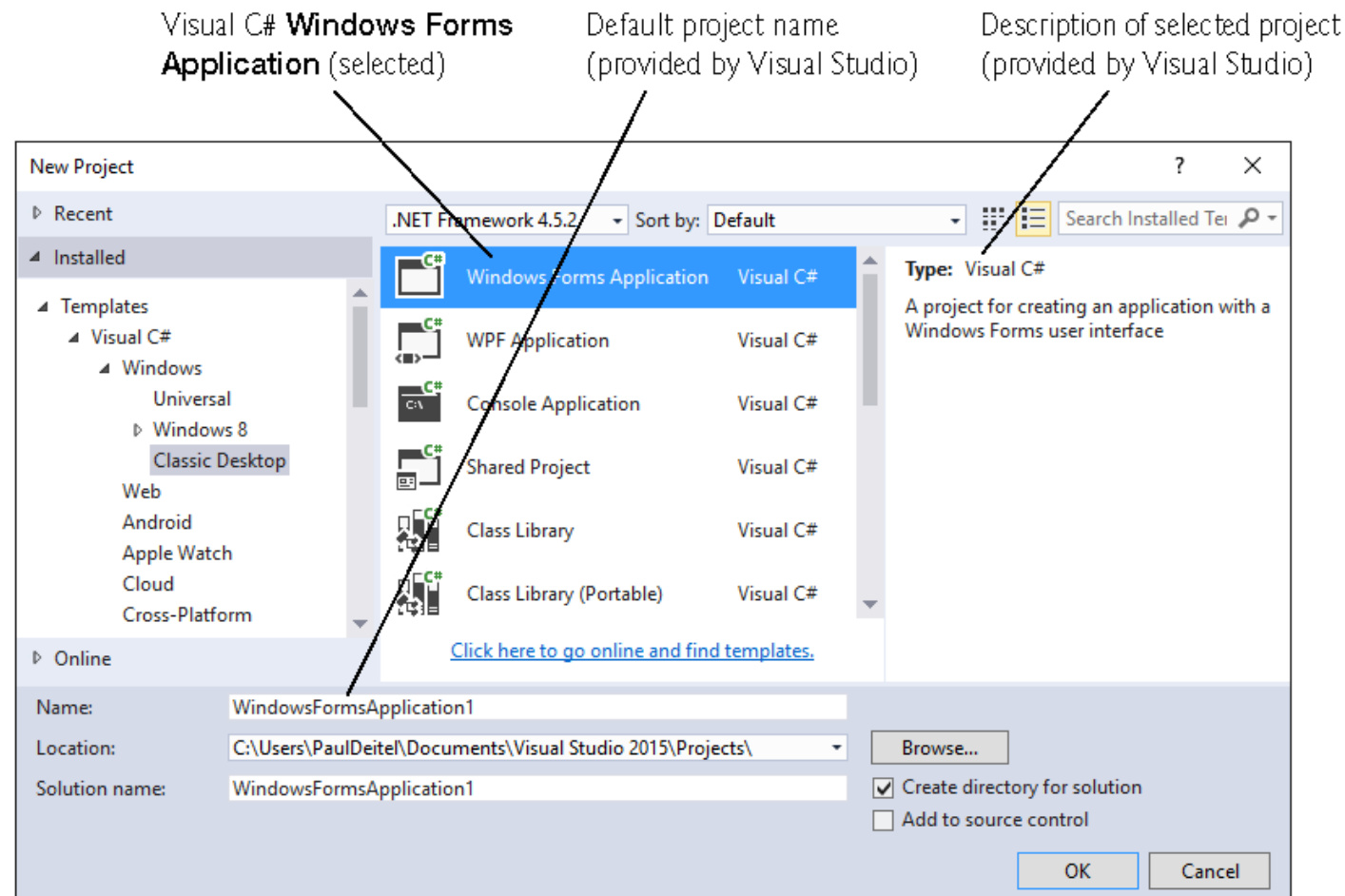
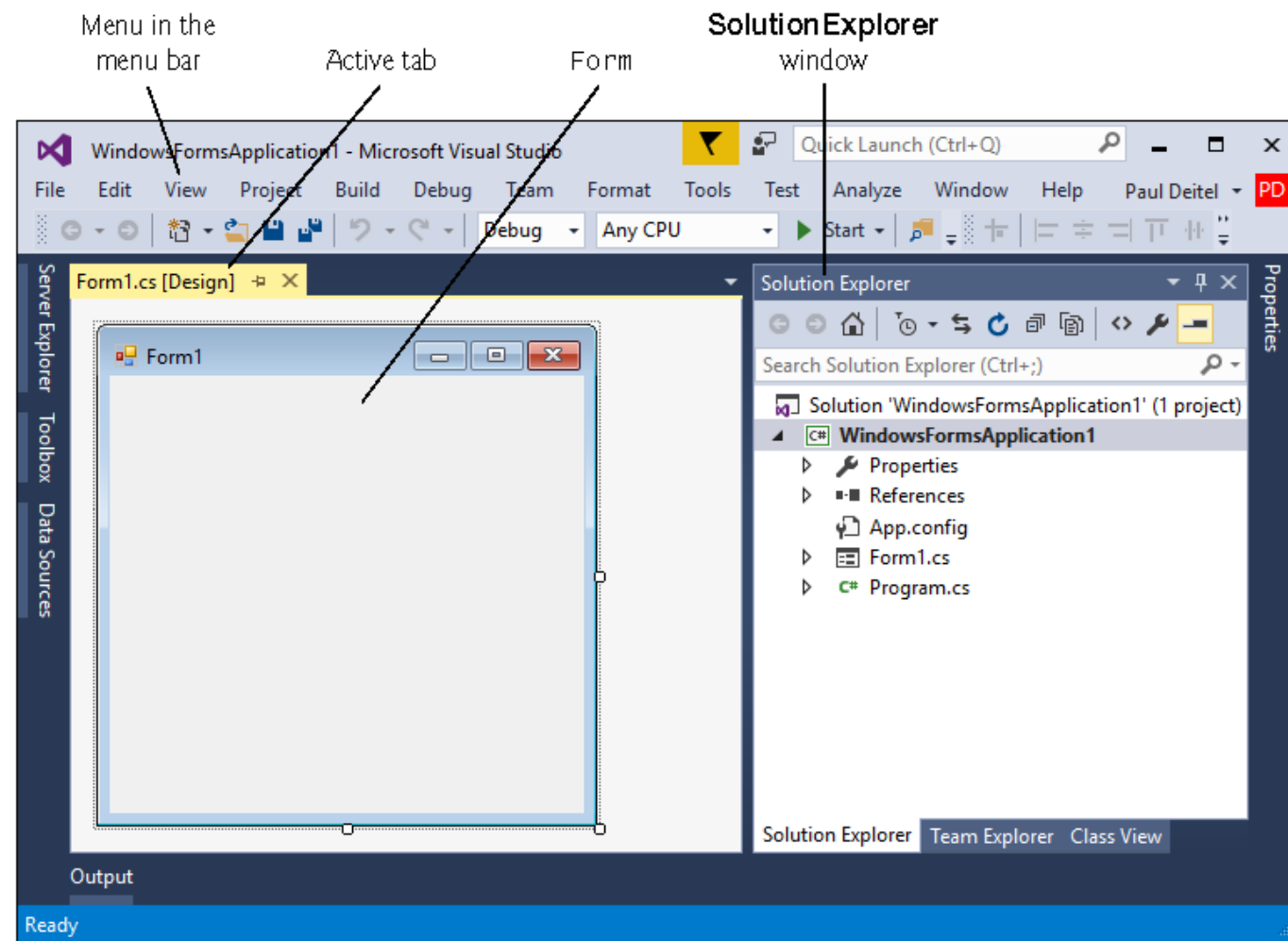


Fig. 2.3 | New Project dialog.

2.2.5 New Project Dialog and Project Templates

- ▶ Visual Studio provides many **templates**—the *project types* that users can create in Visual C# and other languages.
- ▶ A **Windows Forms Application** is an app that executes within a Windows operating system and typically has a **graphical user interface (GUI)**.
 - Users interact with this *visual* part of the app.
- ▶ To create a **Windows Forms Application**, under **Templates** select **Visual C# > Windows > Classic Desktop**, then in the middle column select **Windows Forms Application**.
- ▶ Click **OK** to display the IDE in **Design view** (Fig. 2.4), which contains the features that enable you to create an app's GUI.



2.2.6 Forms and Controls

- ▶ The rectangle in the Design area titled **Form1** (called a **Form**) represents the main window of the Windows Forms app.
- ▶ Each **Form** is an object of class **Form** in the .NET Framework Class Library.
- ▶ Apps can have multiple **Forms** (windows).
- ▶ A **Label** typically contains descriptive text (for example, "Welcome to Visual C#!"), and a **PictureBox** displays an image.
- ▶ Visual Studio has many preexisting controls and other components you can use to build and customize your apps.
- ▶ Collectively, the **Form** and controls make up the app's GUI.

2.2.6 Forms and Controls

- ▶ Each open document's name is listed on a tab. To view a document when multiple documents are open, click its tab.
- ▶ The **active tab** (the tab of the currently displayed document) is highlighted.
- ▶ The active tab's highlight color depends on the Visual Studio theme—the blue theme uses a yellow highlight and the light and dark themes use a blue highlight.

2.3 Menu Bar and Toolbar

- ▶ Commands for managing the IDE and for developing, maintaining and executing apps are contained in menus, which are located on the **menu bar** of the IDE (Fig. 2.5).
 - The set of menus displayed depends on what you're currently doing in the IDE.
- ▶ Menus contain groups of related commands called *menu items* that, when selected, cause the IDE to perform specific actions.
- ▶ The menus depicted in Fig. 2.5 are summarized in Fig. 2.6.

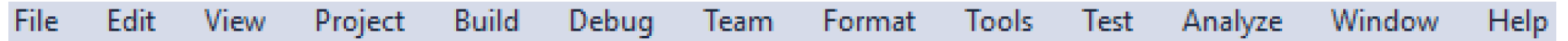
A horizontal menu bar with 13 items: File, Edit, View, Project, Build, Debug, Team, Format, Tools, Test, Analyze, Window, and Help. The items are displayed in a light blue font on a light blue background.

Fig. 2.5 | Visual Studio menu bar.

Menu	Contains commands for
File	Opening, closing, adding and saving projects, as well as printing project data and exiting Visual Studio.
Edit	Editing apps, such as cut, copy, paste, undo, redo, delete, find and select.
View	Displaying IDE windows (for example, Solution Explorer, Toolbox, Properties window) and for adding toolbars to the IDE.
Project	Managing projects and their files.
Build	Turning your app into an executable program.
Debug	Compiling, debugging (that is, identifying and correcting problems in apps) and running apps.
Team	Connecting to a Team Foundation Server—used by development teams that typically have multiple people working on the same app.

Fig. 2.6 | Summary of Visual Studio menus that are displayed when a Form is in **Design** view. (Part 1 of 2.)

Menu	Contains commands for
Format	Arranging and modifying a Form's controls. The Format menu appears <i>only</i> when a GUI component is selected in Design view.
Tools	Accessing additional IDE tools and options for customizing the IDE.
Test	Performing various types of automated testing on your app.
Analyze	Locating and reporting violations of the .NET Framework Design Guidelines (https://msdn.microsoft.com/library/ms229042).
Window	Hiding, opening, closing and displaying IDE windows.
Help	Accessing the IDE's help features.

Fig. 2.6 | Summary of Visual Studio menus that are displayed when a Form is in **Design** view. (Part 2 of 2.)

2.3 Menu Bar and Toolbar

- ▶ You can access many common menu commands from the **toolbar** (Fig. 2.7), which contains **icons** that graphically represent commands.
- ▶ By default, the standard toolbar is displayed when you run Visual Studio for the first time—it contains icons for the most commonly used commands, such as opening a file, saving files and running apps.
 - The icons that appear on the standard toolbar may vary, depending on the version of Visual Studio you're using.

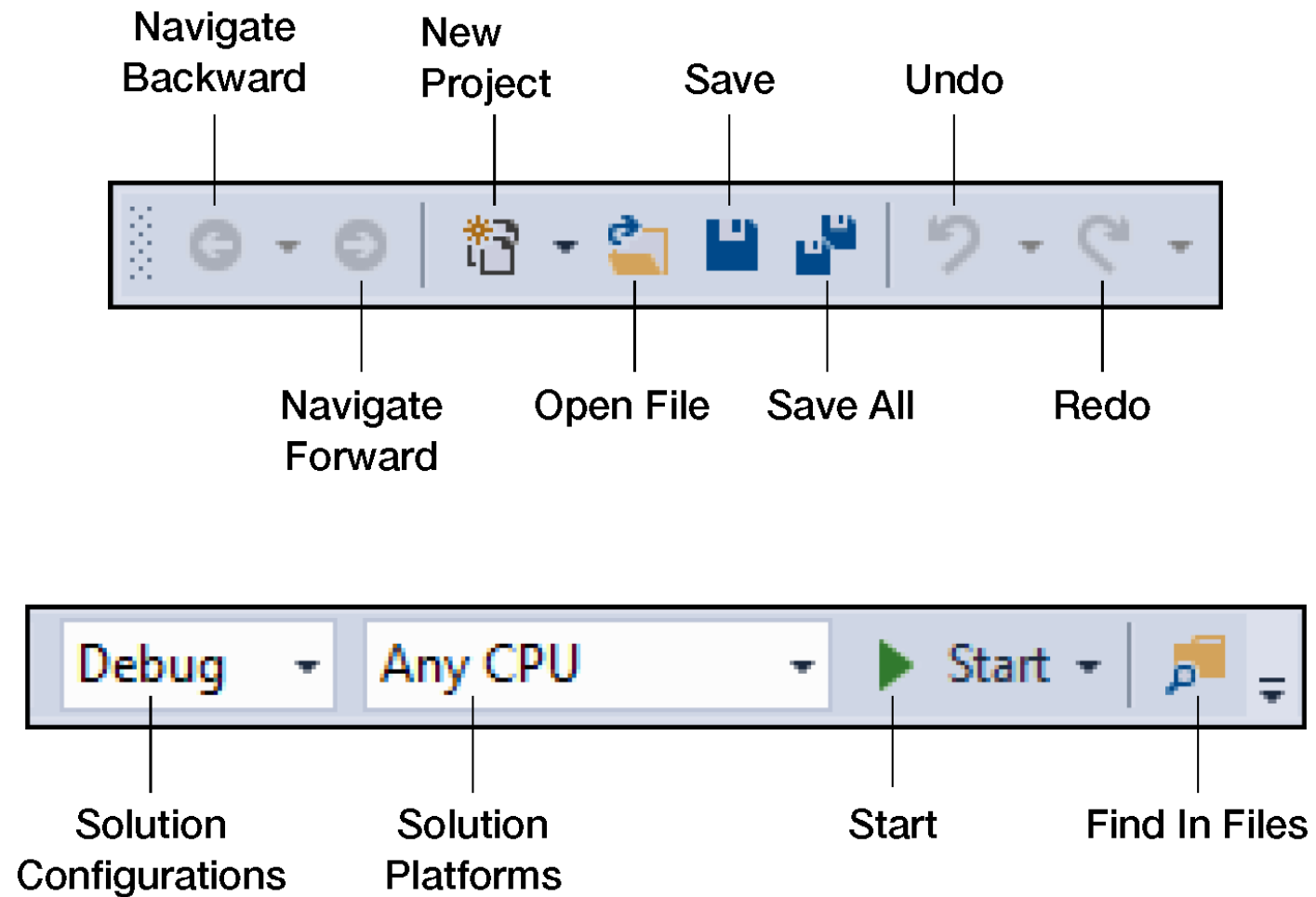
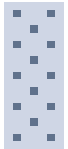


Fig. 2.7 | Standard Visual Studio toolbar.

2.3 Menu Bar and Toolbar

- ▶ You can customize which toolbars are displayed by selecting **View > Toolbars** then selecting a toolbar from the list in Fig. 2.8.
- ▶ Each toolbar you select is displayed with the other toolbars at the top of the Visual Studio window.
- ▶ You move a toolbar by dragging its handle



- ▶ To execute a command via the toolbar, click its icon.
- ▶ *Hovering* the mouse pointer over an icon highlights it and displays a description of the icon called a **tool tip** (Fig. 2.9)

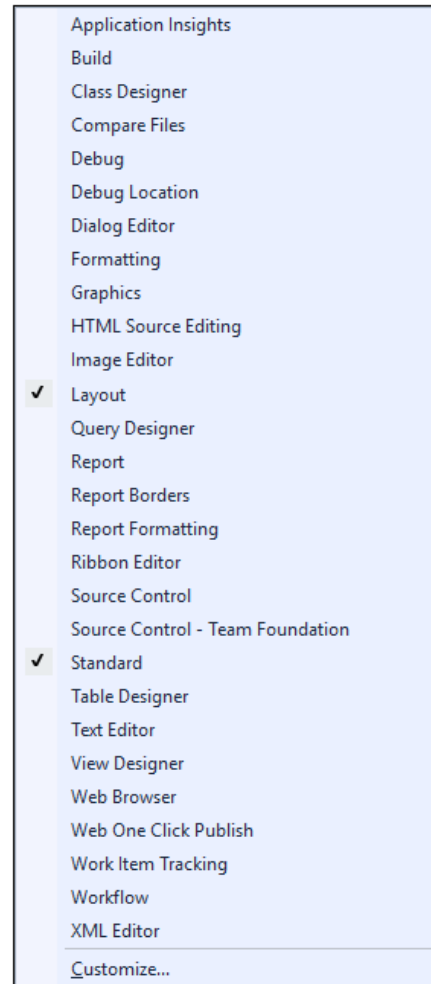
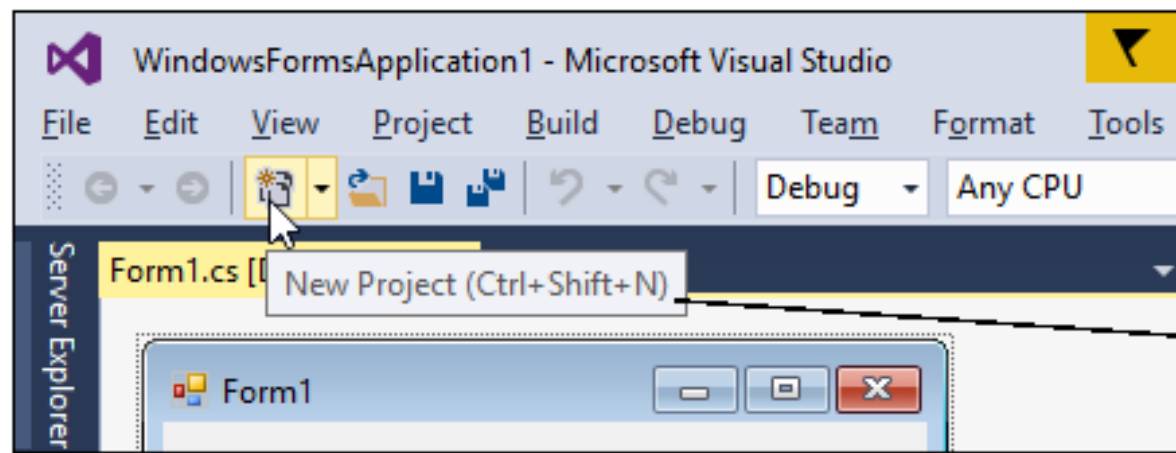


Fig. 2.8 | List of toolbars that can be added to the top of the IDE.



Tool tip appears when you place the mouse pointer on an icon

Fig. 2.9 | Tool tip for the **New Project** button.

2.4 Navigating the Visual Studio IDE

- ▶ The IDE provides windows for accessing project files and customizing controls. Each can be accessed by selecting its name in the **View** menu.
- ▶ Visual Studio provides an **auto-hide** space-saving feature.
 - When auto-hide is enabled for a window, a tab containing the window's name appears along the IDE window's left, right or bottom edge (Fig. 2.10).
 - Clicking the name of an auto-hidden window displays that window (Fig. 2.11).
 - Clicking the name again (or clicking outside) hides the window.
- ▶ To “pin down” a window (that is, to disable auto-hide and keep the window open), click the pin icon.
 - When auto-hide is enabled, the pin icon is horizontal
 - When a window is “pinned down,” the pin icon is vertical

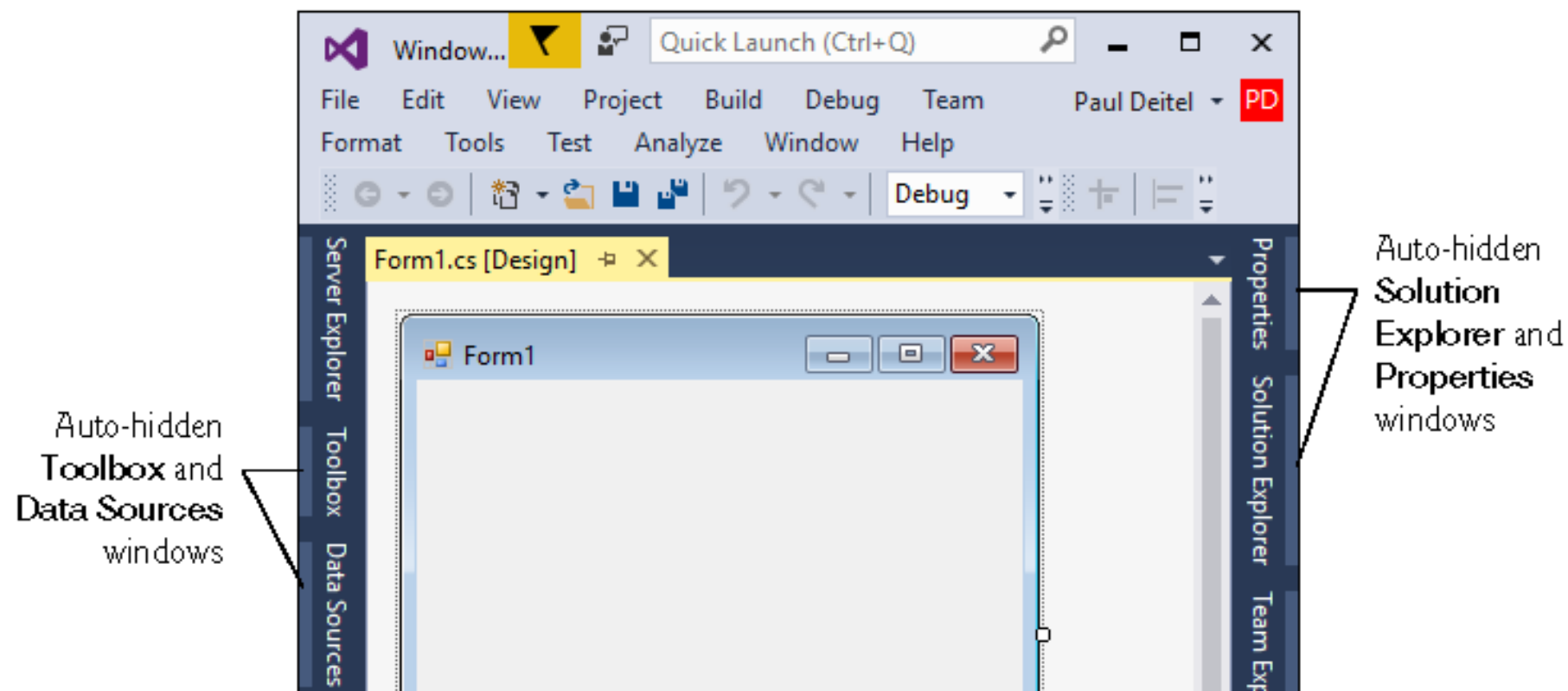


Fig. 2.10 | Auto-hide feature demonstration.

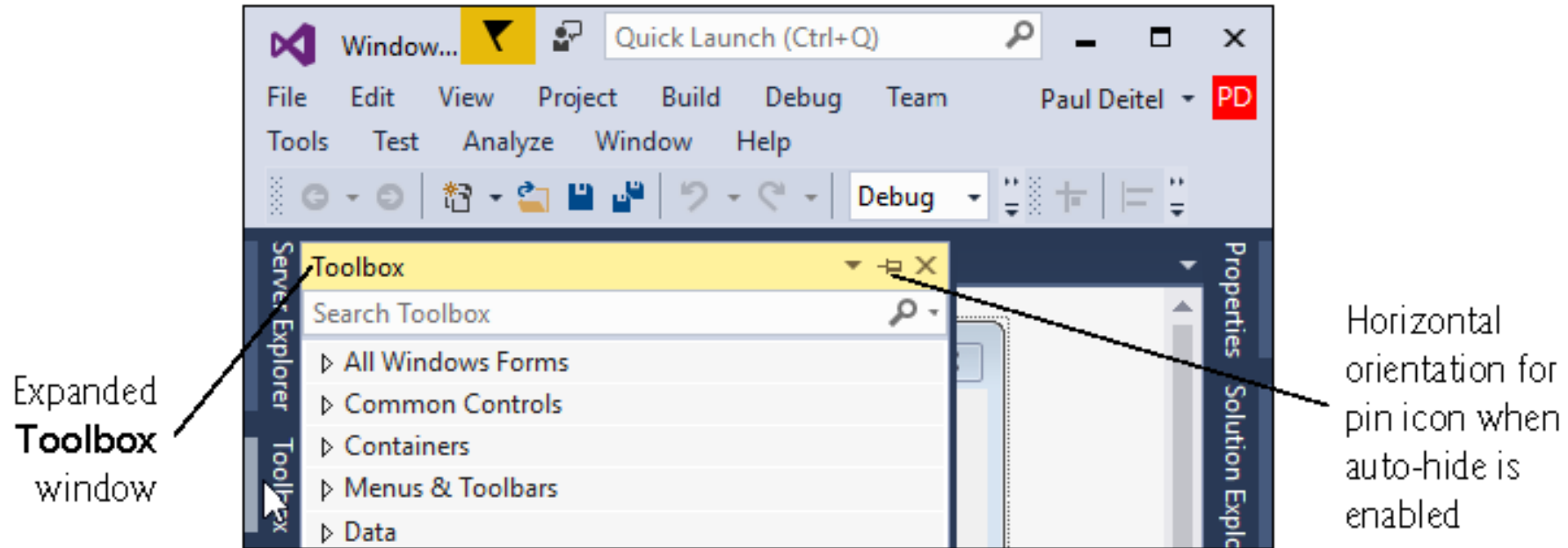
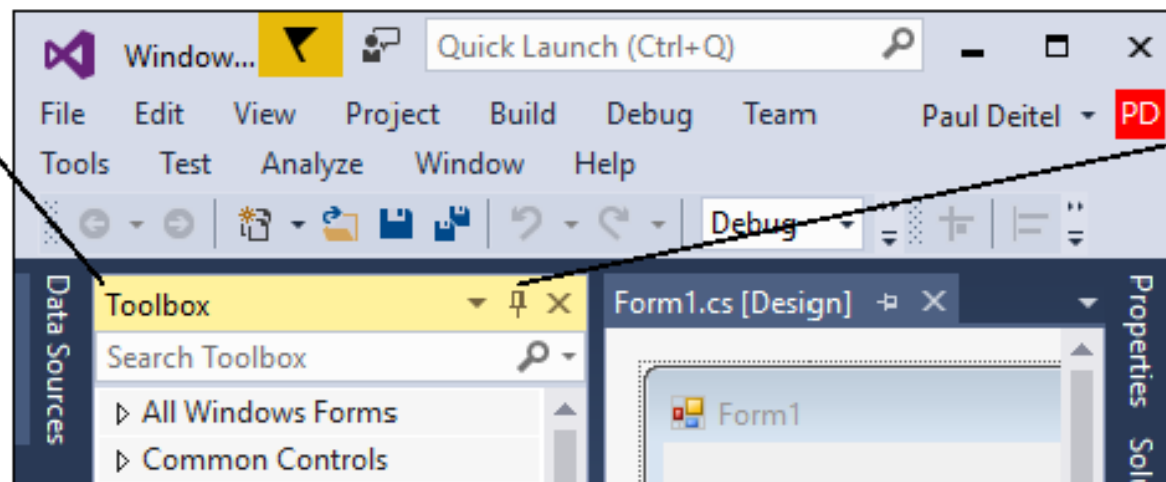


Fig. 2.11 | Displaying the hidden **Toolbox** window when auto-hide is enabled.

Toolbox
“pinned down”



Vertical
orientation for
pin icon when
window is
“pinned down”

Fig. 2.12 | Disabling auto-hide—“pinning down” a window.

2.4.1 Solution Explorer

- ▶ The **Solution Explorer** window (Fig. 2.13) provides access to all of a solution's files.
- ▶ If it's not shown in the IDE, select **View > Solution Explorer**.
- ▶ When you open a new or existing solution, the Solution Explorer displays the solution's contents.

2.4.1 Solution Explorer

- ▶ The solution's **startup project** (shown in **bold** in the Solution Explorer) is the one that runs when you select **Debug > Start Debugging** (or press *F5*) or select **Debug > Start Without Debugging** (or press *Ctrl + F5* key).
- ▶ For a single-project solution, the startup project is the only project.
- ▶ When you create an app for the first time, the Solution Explorer window appears as shown in Fig. 2.13.
- ▶ The Visual C# file that corresponds to the **Form** shown in Fig. 2.4 is named **Form1.cs** (selected in Fig. 2.13).
- ▶ Visual C# files use the **.cs** file-name extension, which is short for “C#.”

2.4.1 Solution Explorer

- ▶ By default, the IDE displays only files that you may need to edit—other files that the IDE generates are hidden.
- ▶ The Solution Explorer window includes a toolbar that contains several icons.
- ▶ Clicking the **Show All Files icon** (Fig. 2.13) displays all the solution's files, including those generated by the IDE.
- ▶ Clicking the arrow to the left of a node *expands* or *collapses* that node.
- ▶ Click the arrow to the left of **References** to display items grouped under that heading (Fig. 2.14).
- ▶ Click the arrow again to collapse the tree.

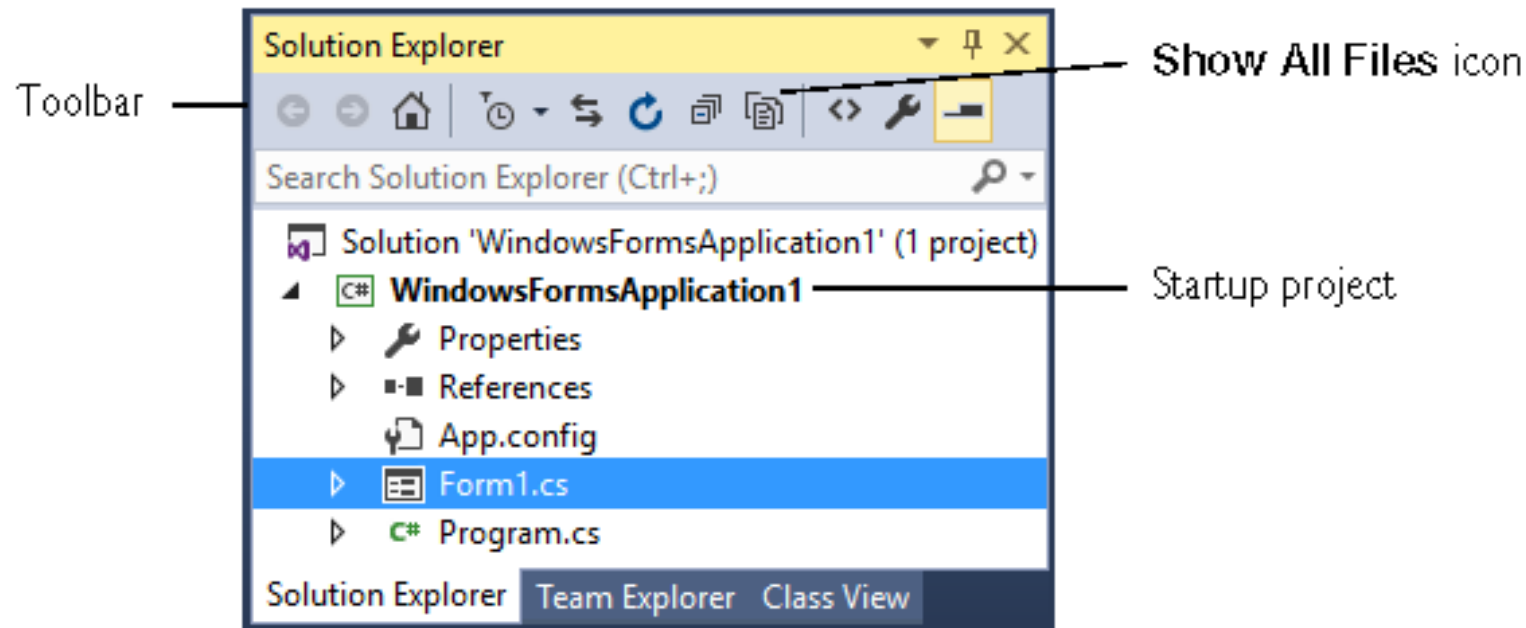


Fig. 2.13 | Solution Explorer window showing the **WindowsFormsApplication1** project.

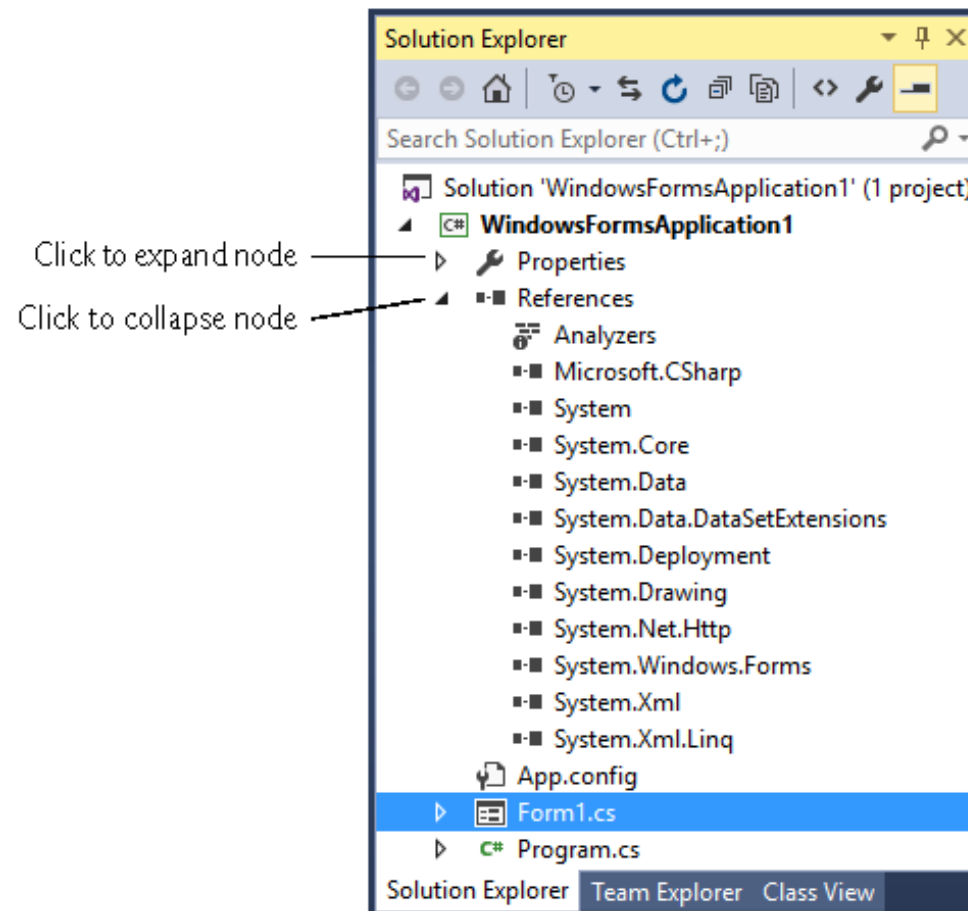


Fig. 2.14 | Solution Explorer with the **References** node expanded.

2.4.2 Toolbox

- ▶ To display the **Toolbox** window, select **View > Toolbox**.
 - Contains the controls used to customize Forms (Fig. 2.15).
- ▶ With visual app development, you can “drag and drop” controls onto the Form and the IDE will write the code that creates the controls for you.
- ▶ The Toolbox groups the prebuilt controls into categories—**All Windows Forms, Common Controls, Containers, Menus & Toolbars, Data, Components, Printing, Dialogs, Reporting, WPF Interoperability and General**.

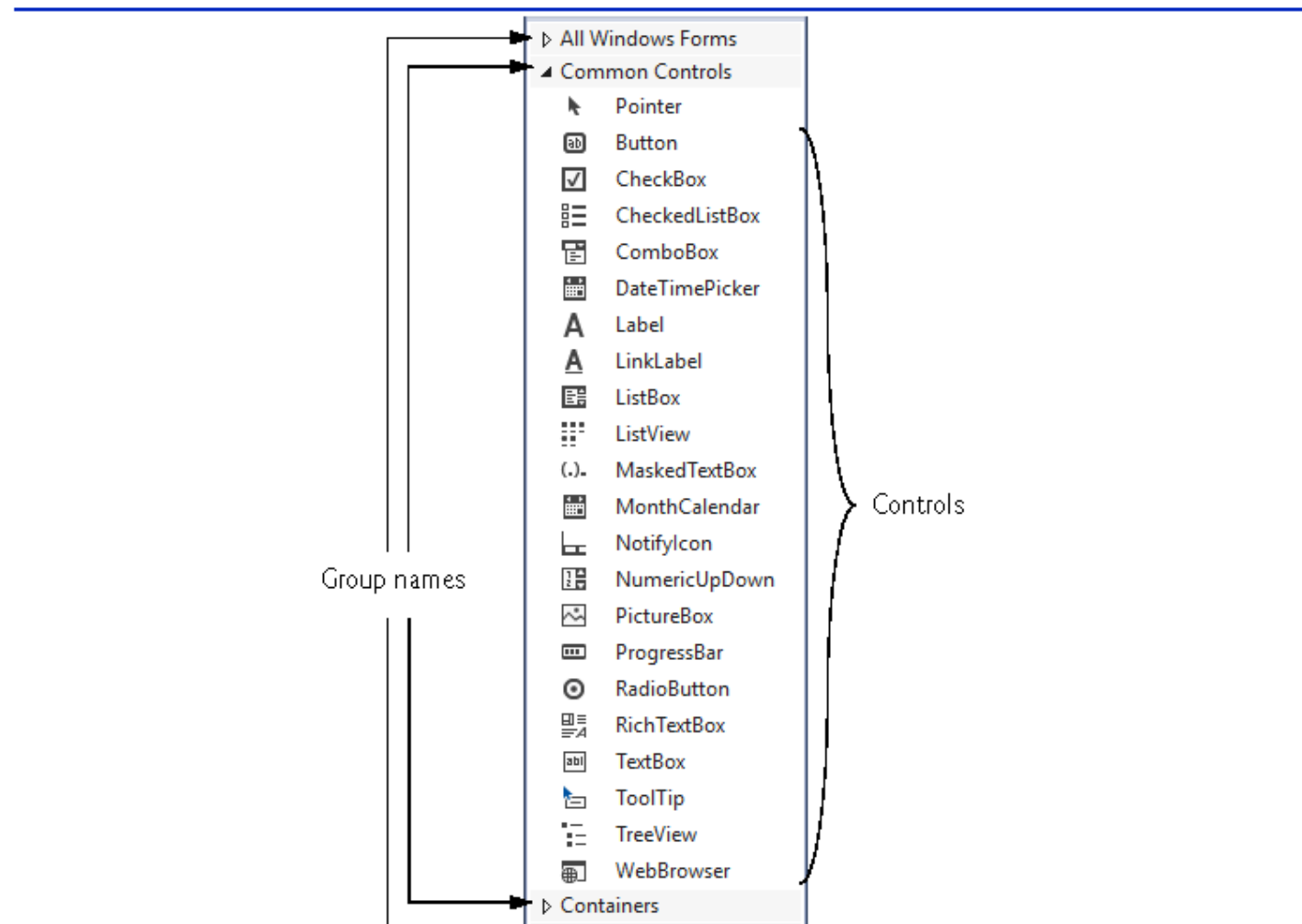


Fig. 2.15 | **Toolbox** window displaying controls for the **Common Controls** group.

2.4.3 Properties Window

- ▶ If the **Properties** window is not displayed below the Solution Explorer, select **View > Properties Window** to display it.
- ▶ The **Properties window** contains the properties for the currently selected **Form**, control or file in the IDE.
- ▶ **Properties** specify information about the **Form** or control.
- ▶ When you select a property, its description is displayed at the bottom of the **Properties** window.

2.4.3 Properties Window

- ▶ Fig. 2.16 shows **Form1's Properties** window—you can view by clicking anywhere in the **Form1.cs [Design]** window.
- ▶ The left column lists the **Form's** properties
- ▶ The right column displays the current value of each property.
- ▶ You can sort the properties either
 - *alphabetically* (by clicking the **Alphabetical icon**) or
 - *categorically* (by clicking the **Categorized icon**).
- ▶ The **Properties** window is crucial to visual app development.

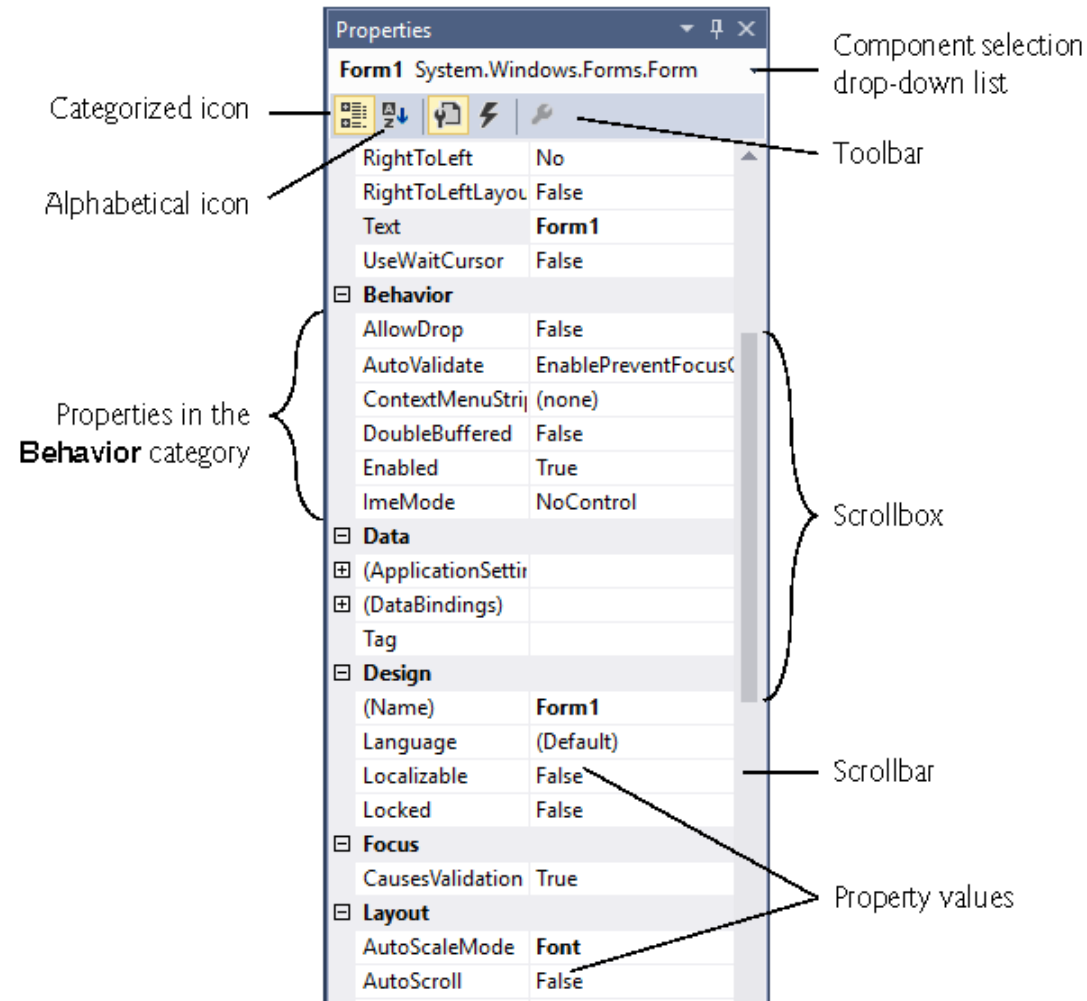


Fig. 2.16 | Properties window.



2.5 Help Menu and Context-Sensitive Help

- ▶ Microsoft provides extensive help documentation via the **Help menu**, which is an excellent way to get information quickly about Visual Studio, Visual C# and more.
- ▶ Visual Studio provides **context-sensitive help** pertaining to the “current content” (that is, the items around the location of the mouse cursor).
- ▶ To use context-sensitive help, click an item, then press the *F1* key.

2.6 Visual Programming: Creating a Simple App that Displays Text and an Image



- ▶ Next, we create an app that displays the text "Welcome to C# Programming!" and an image of the Deitel & Associates bug mascot.
- ▶ The app consists of a Form that uses a Label and a PictureBox. shows the final app executing.
- ▶ The app and the bug image are available with this chapter's examples.

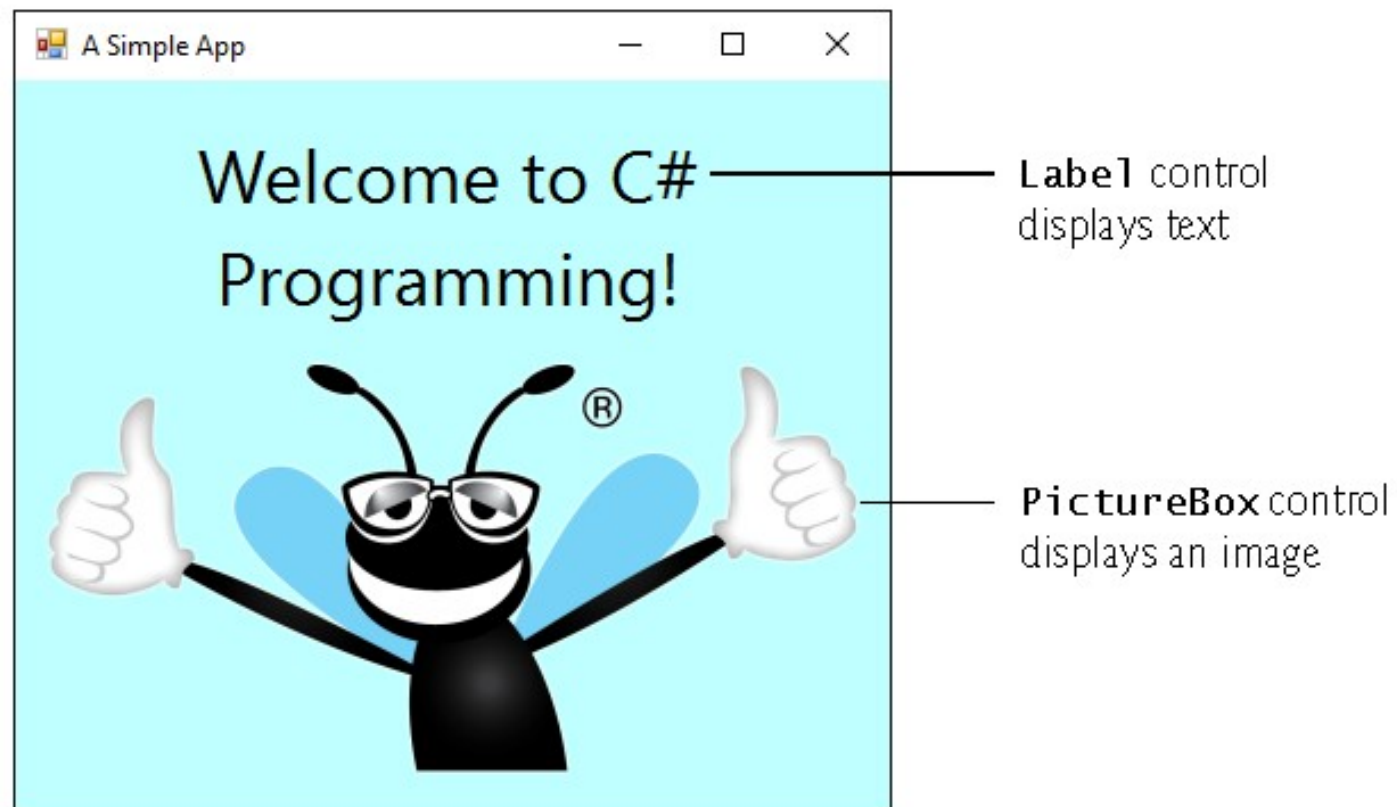
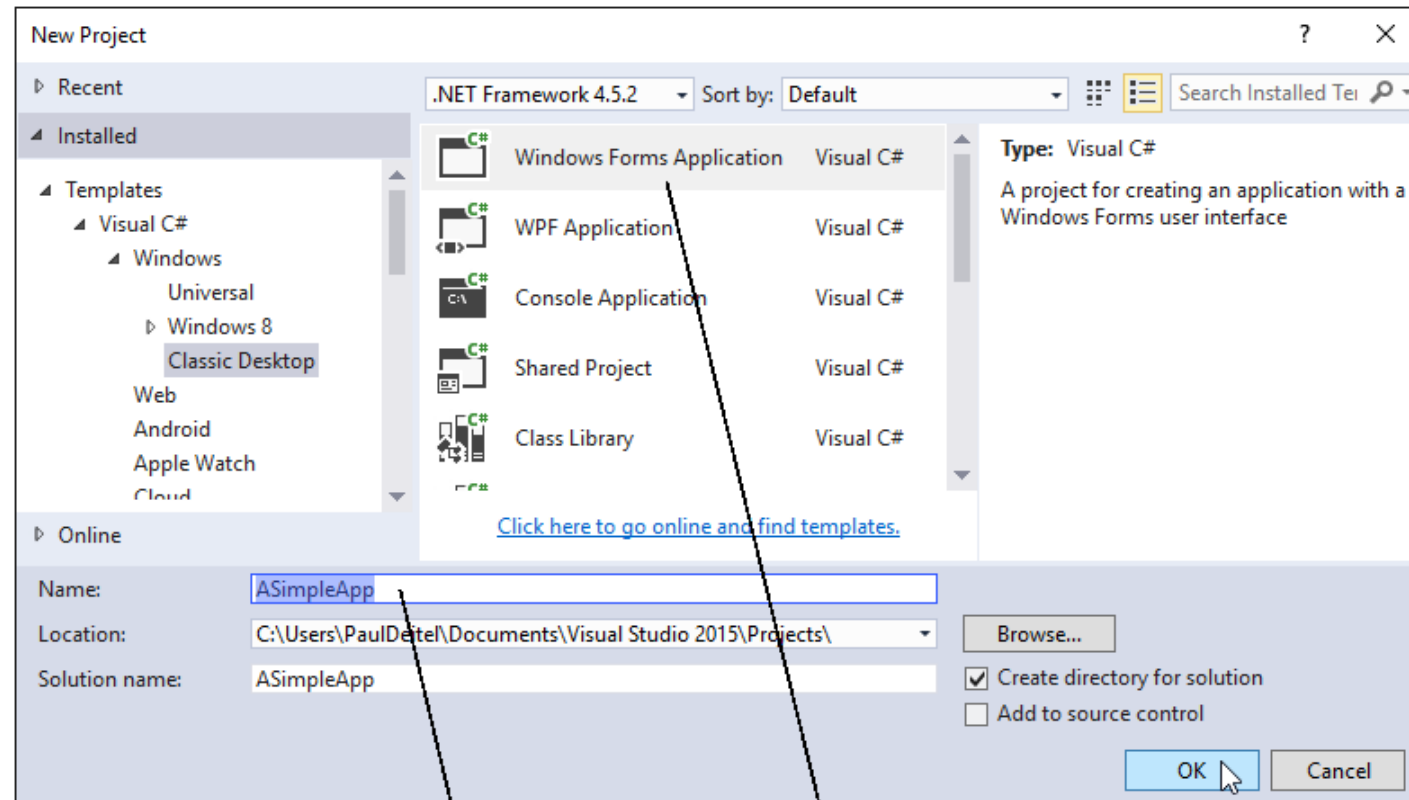


Fig. 2.17 | Simple app executing.



Type the project
name here

Select the **Windows Forms
Application** template

Fig. 2.18 | New Project dialog.

2.6 Visual Programming: Creating a Simple App that Displays Text and an Image



- ▶ The text in the Form's title bar is determined by the Form's **Text** property.

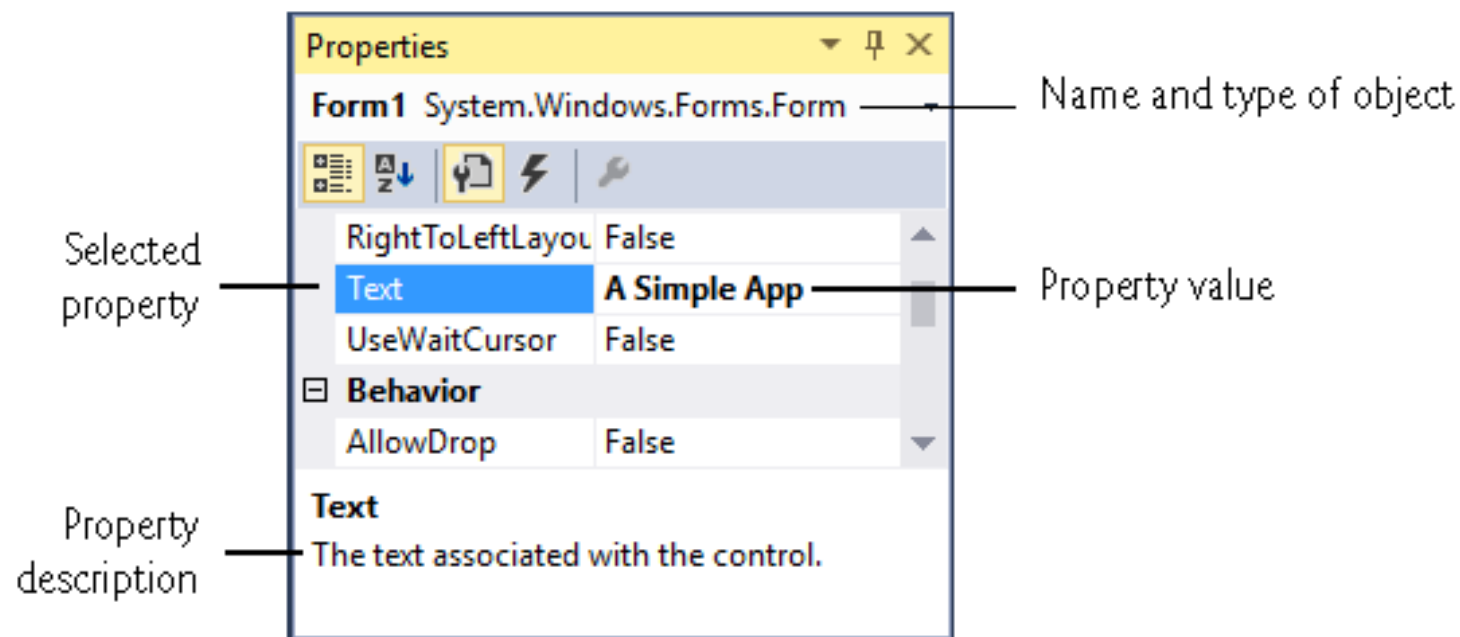


Fig. 2.19 | Setting the **Form's Text** property in the **Properties** window.

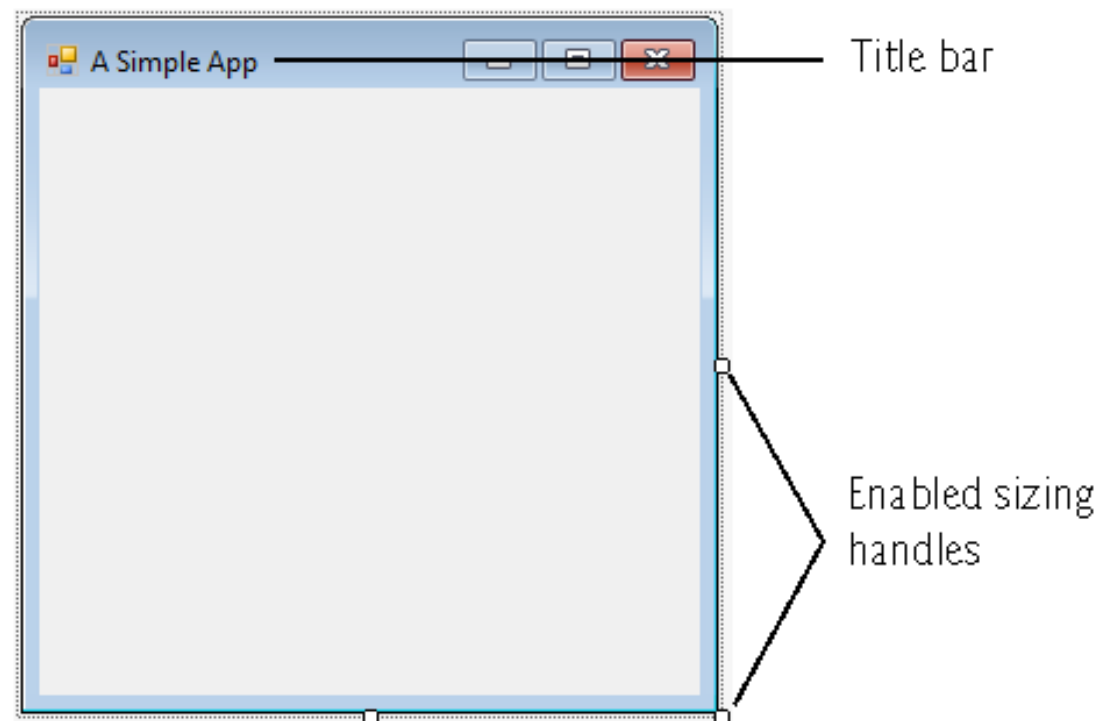


Fig. 2.20 | Form with updated title-bar text and enabled sizing handles.

2.6 Visual Programming: Creating a Simple App that Displays Text and an Image



- ▶ The **Form**'s size is specified in pixels (that is, dots on the screen).
- ▶ By default, a **Form** is 300 pixels wide and 300 pixels tall.
- ▶ You can resize the **Form** by dragging one of its **sizing handles** (the small white squares that appear around the **Form**, as shown in).
- ▶ As you drag the mouse, the IDE's status bar (at the bottom of the IDE) shows the current width and height in pixels.
- ▶ You also can do this via the **Form**'s **Size** property in the **Properties** window.

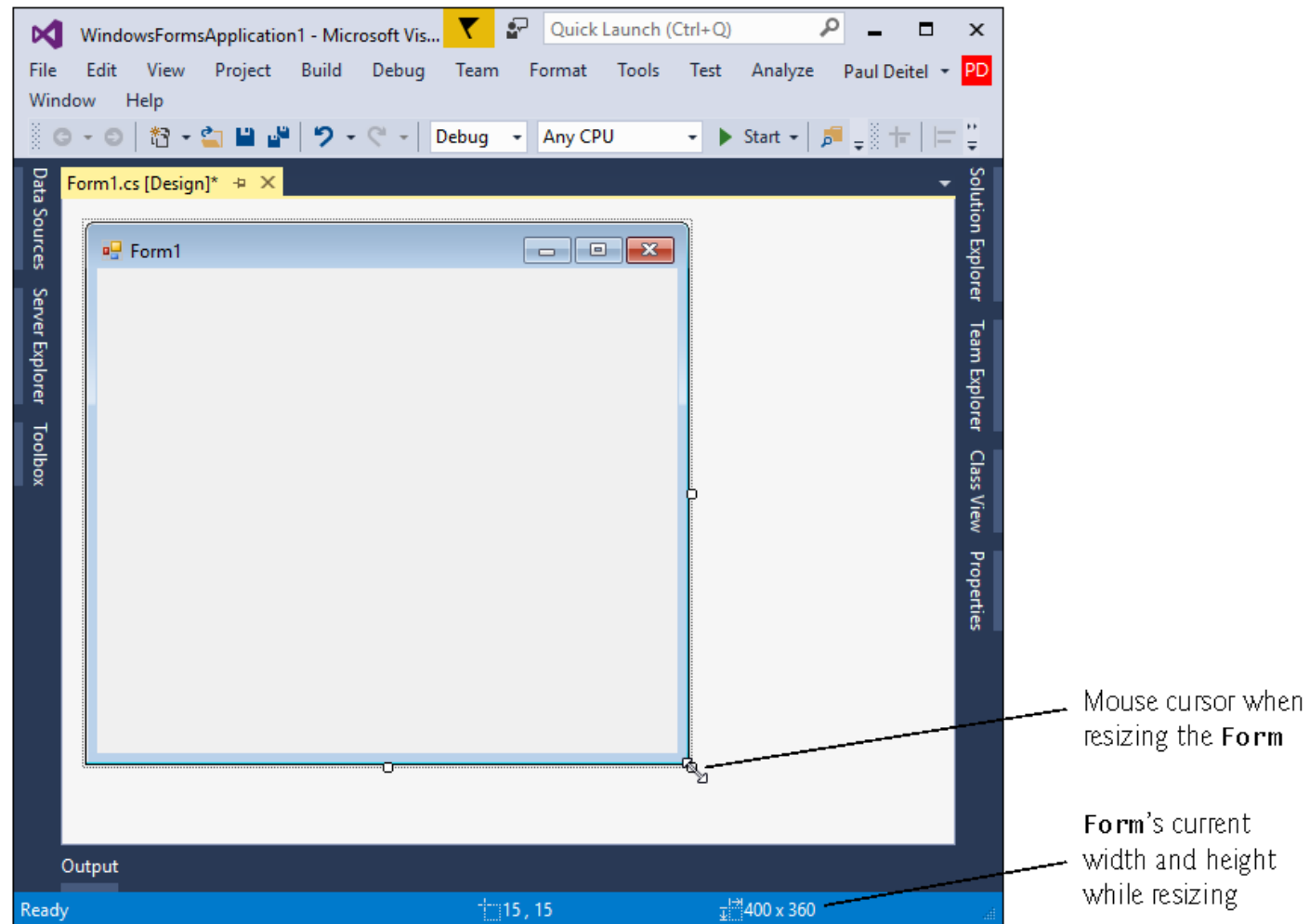


Fig. 2.21 | Resizing the Form.

2.6 Visual Programming: Creating a Simple App that Displays Text and an Image



- ▶ The **BackColor** property specifies a Form's or control's background color.
- ▶ Clicking **BackColor** in the **Properties** window causes a down-arrow button to appear next to the value of the property.
- ▶ Clicking the down-arrow button displays other options, which vary depending on the property.

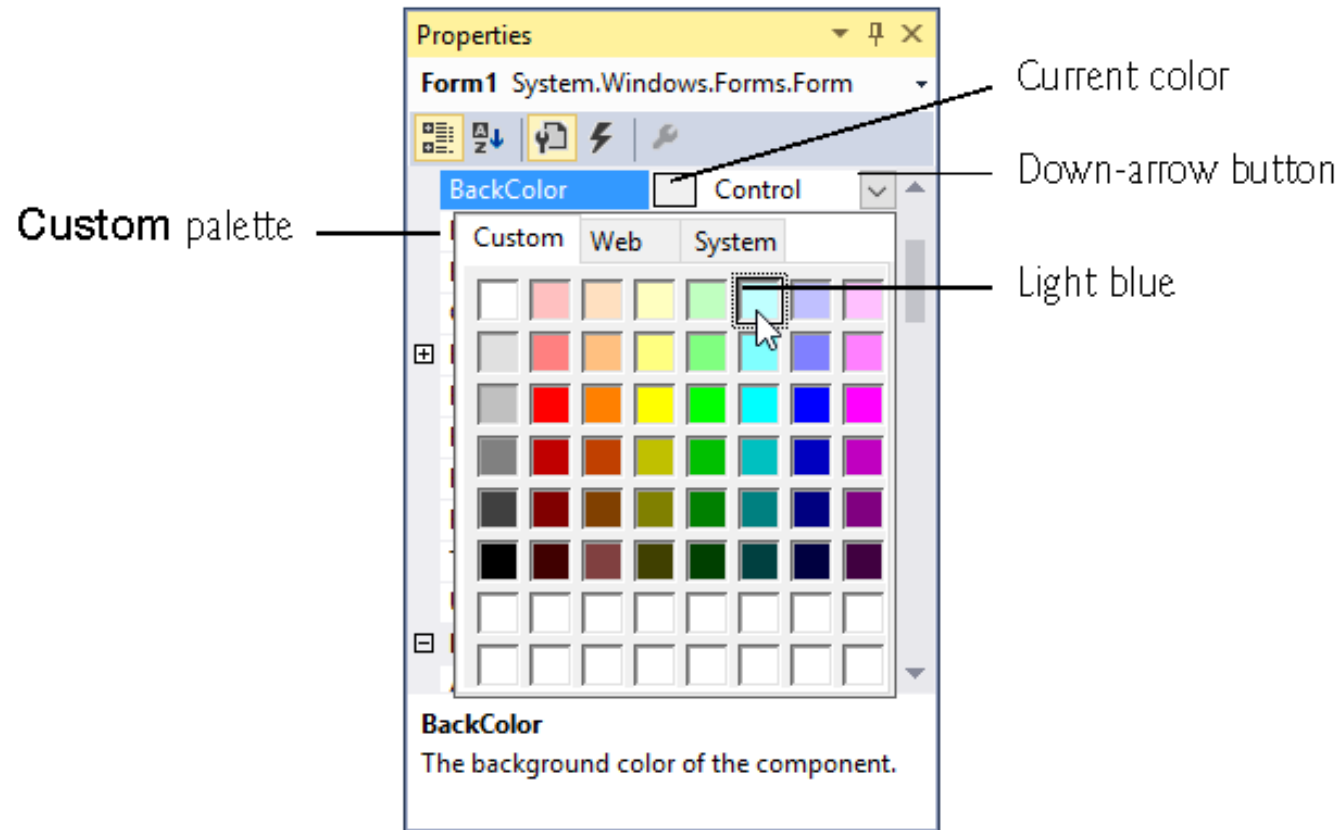


Fig. 2.22 | Changing the Form's BackColor property.

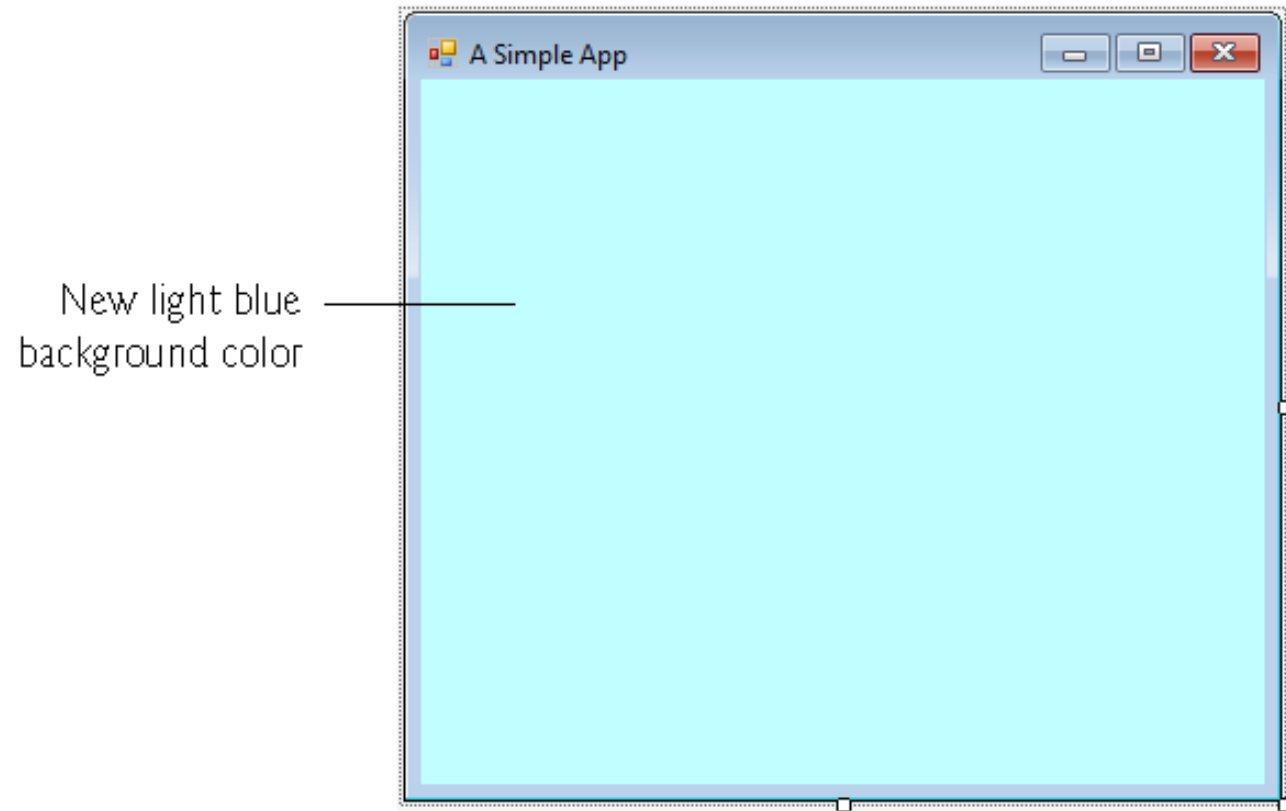


Fig. 2.23 | Form with new `BackColor` property applied.

2.6 Visual Programming: Creating a Simple App that Displays Text and an Image



- ▶ Typical controls we use are located in the **Toolbox's Common Controls** group, and also can be found in the **All Windows Forms** group.
- ▶ Each `Label` you add to the `Form` is an object of class `Label` from the .NET Framework Class Library.
- ▶ By default, a `Label's BackColor` is the same as the `Form's`.

Label control

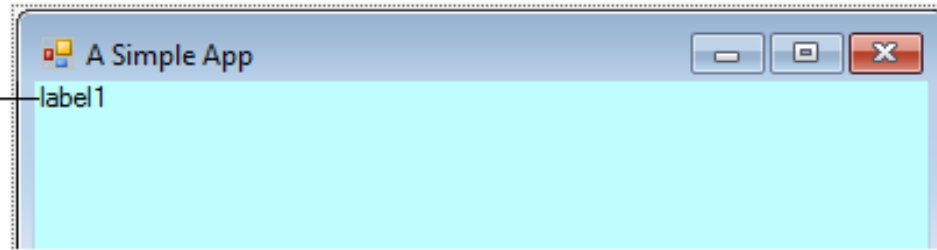


Fig. 2.24 | Adding a Label to the Form.

2.6 Visual Programming: Creating a Simple App that Displays Text and an Image



- ▶ Form and Label each have their own **Text** property
- ▶ By default, the **AutoSize** property of the Label is set to **True** so the Label can update its own size to fit all of its text.
- ▶ Setting **AutoSize** property to **False** allows you to change the Label's size using the sizing handles.
- ▶ You can center a control horizontally by selecting **Format > Center In Form > Horizontally**.

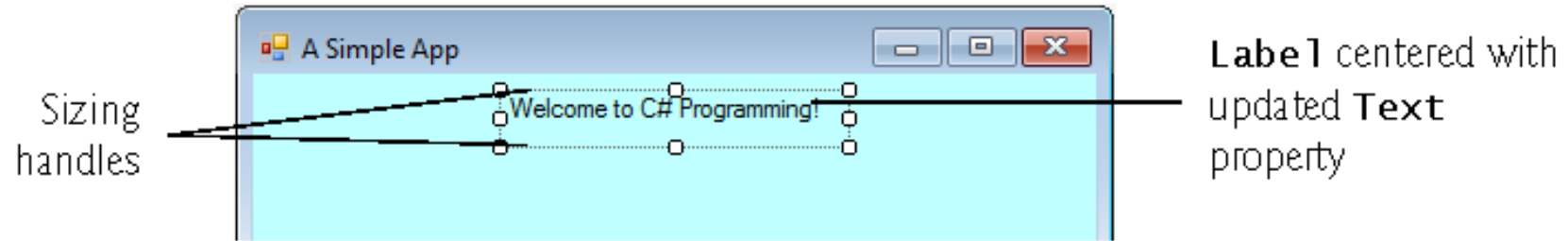


Fig. 2.25 | GUI after the **Form** and **Label1** have been customized.

2.6 Visual Programming: Creating a Simple App that Displays Text and an Image



- ▶ ***Step 8: Setting the Label's Font Size***
- ▶ Select the value of a Label's **Font** property, then click the ellipsis button to display the Font dialog (Fig. 2.27).

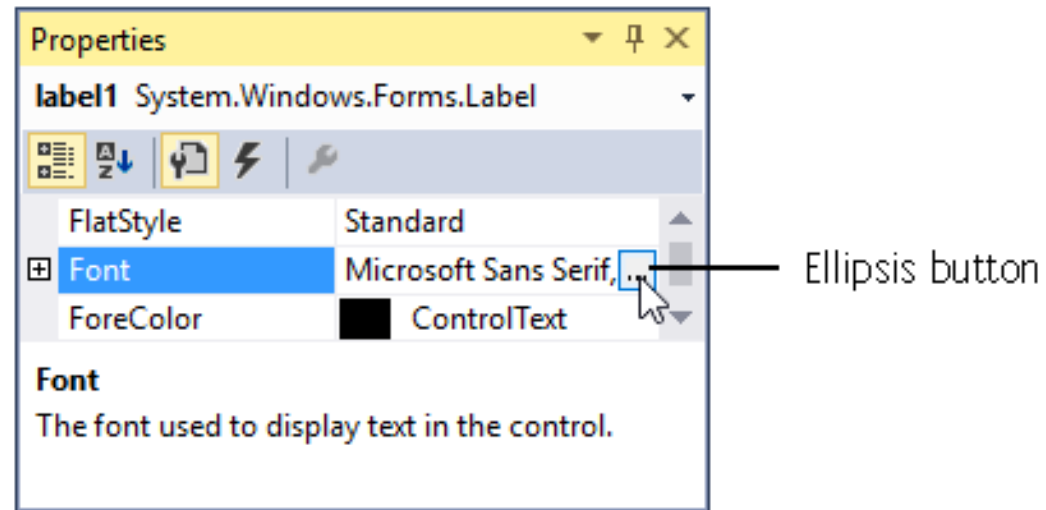


Fig. 2.26 | Properties window displaying the `Label1`'s `Font` property.

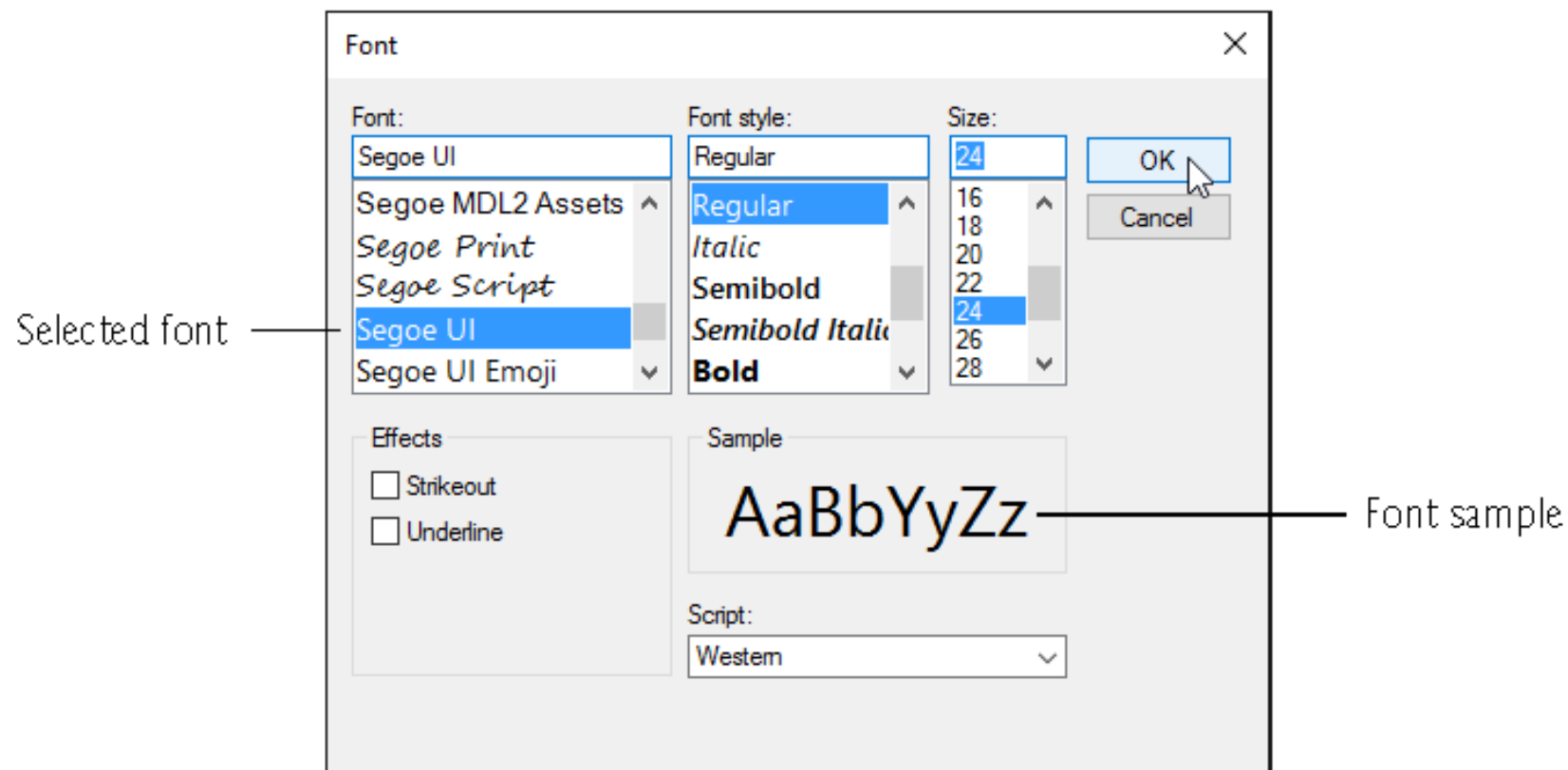


Fig. 2.27 | **Font** dialog for selecting fonts, styles and sizes.

2.6 Visual Programming: Creating a Simple App that Displays Text and an Image



- ▶ Select the `Label`'s **`TextAlign`** property to display a three-by-three grid of buttons representing alignment choices.
- ▶ The position of each button corresponds to where the text appears in the `Label`.
- ▶

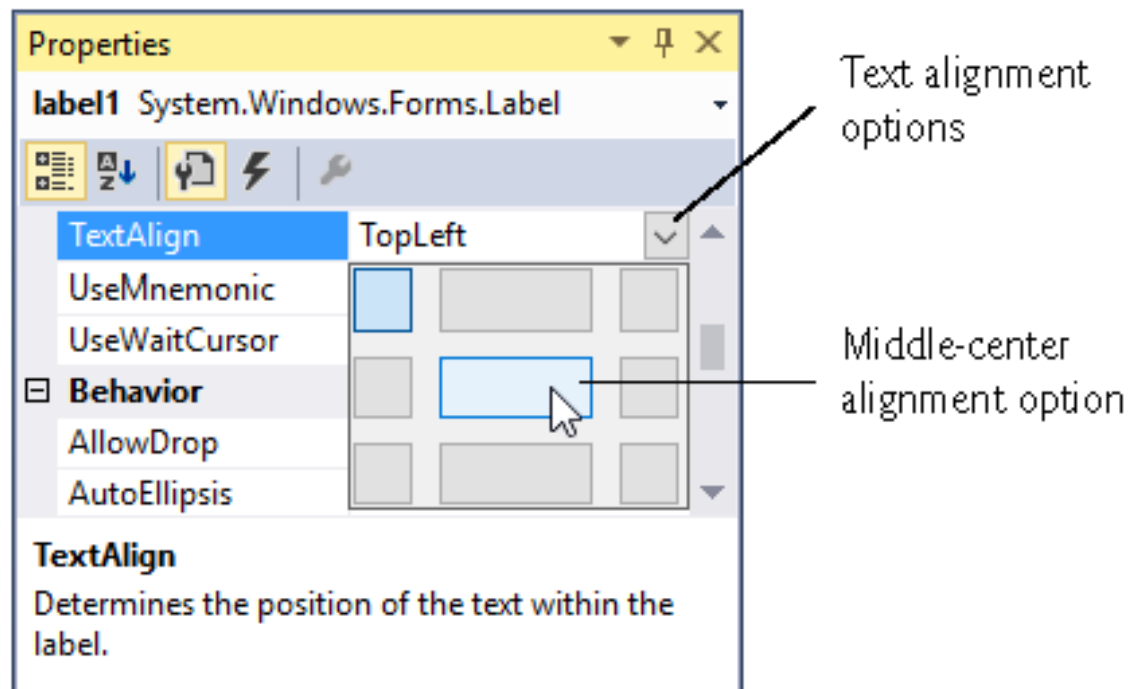


Fig. 2.28 | Centering the `Label`'s text.

2.6 Visual Programming: Creating a Simple App that Displays Text and an Image



- ▶ The `PictureBox` control displays images.
- ▶ Each `PictureBox` you add to the Form is an object of class `PictureBox` from the .NET Framework Class Library.

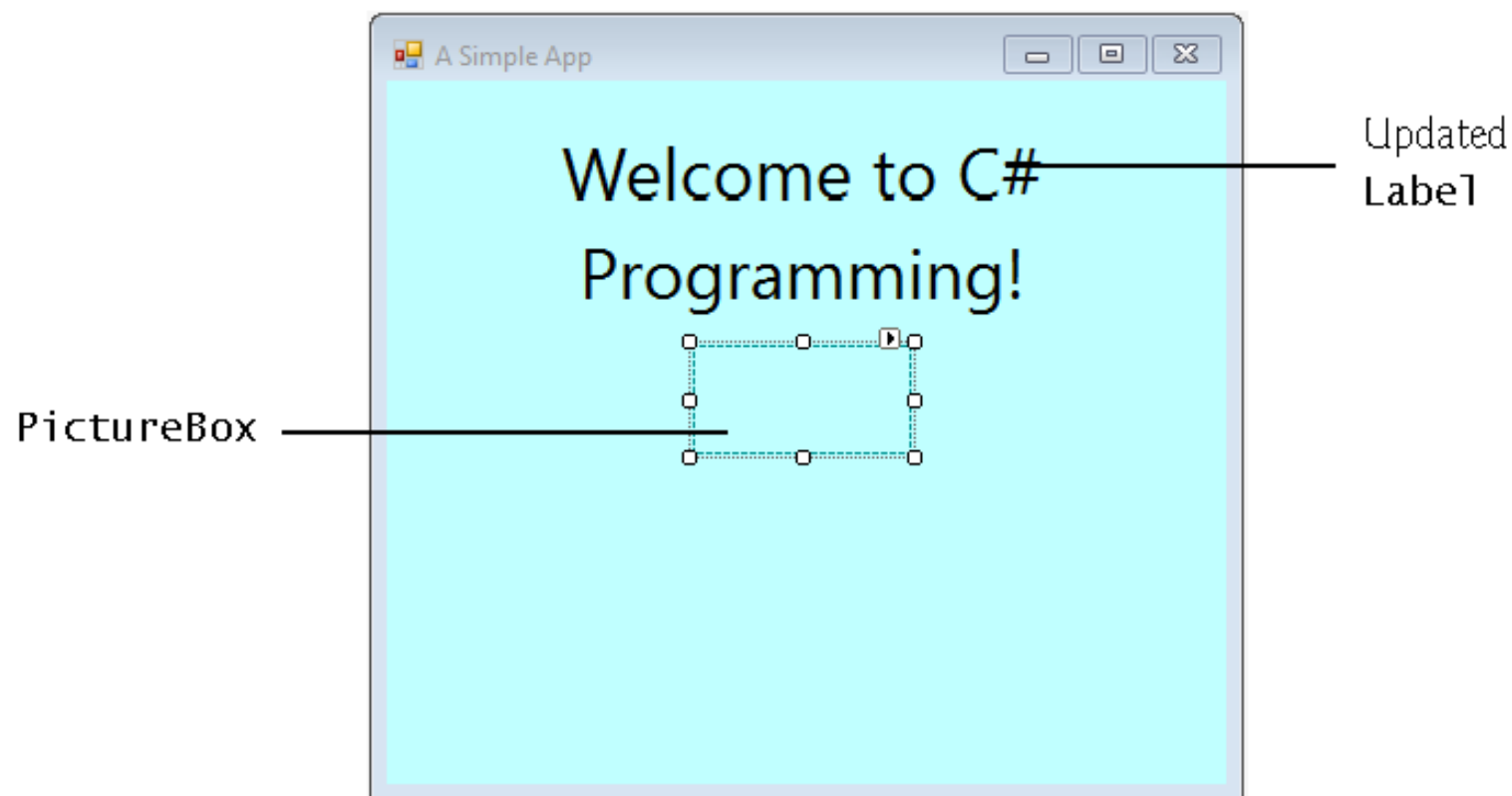


Fig. 2.29 | Inserting and aligning a PictureBox.

2.6 Visual Programming: Creating a Simple App that Displays Text and an Image



- ▶ The **Image property** specifies the image to display in a **PictureBox**.
- ▶ To scale the image to fit in the **PictureBox**, change the **SizeMode property** to **StretchImage**.

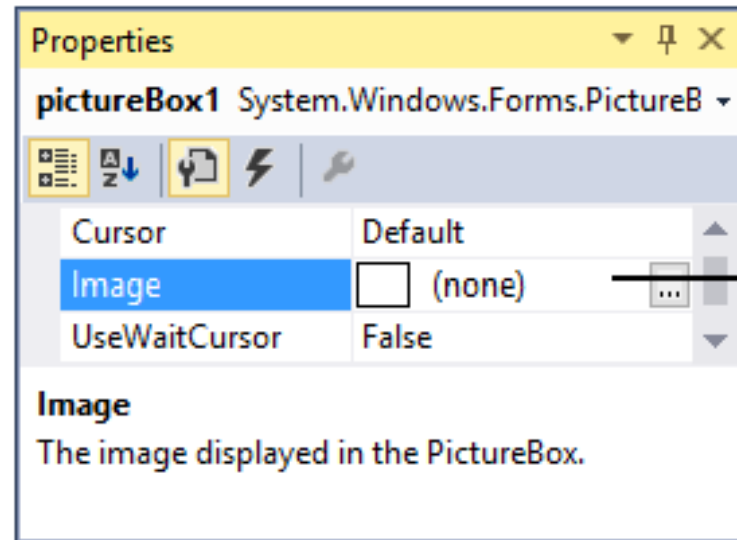


Image property value
(no image selected)

Fig. 2.30 | Image property of the `PictureBox`.

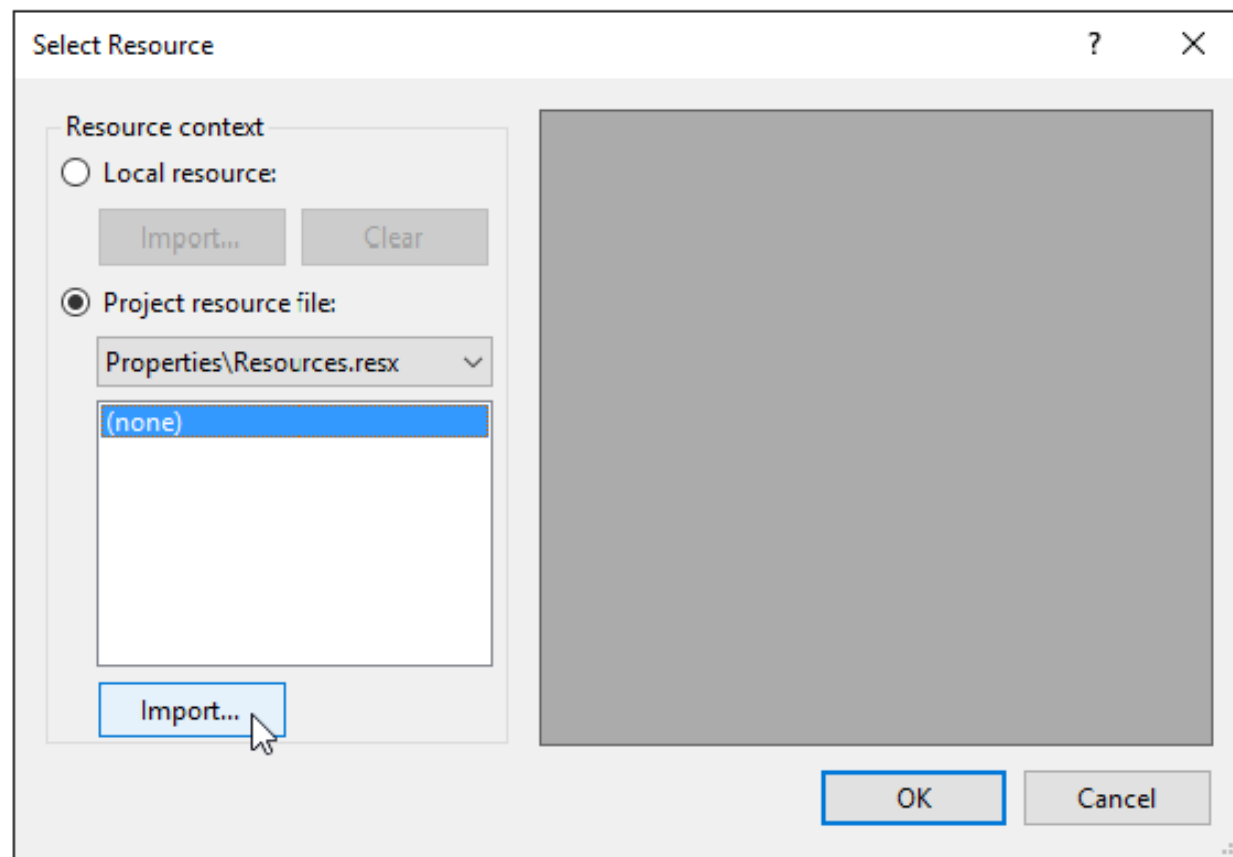


Fig. 2.31 | **Select Resource** dialog to select an image for the `PictureBox`.

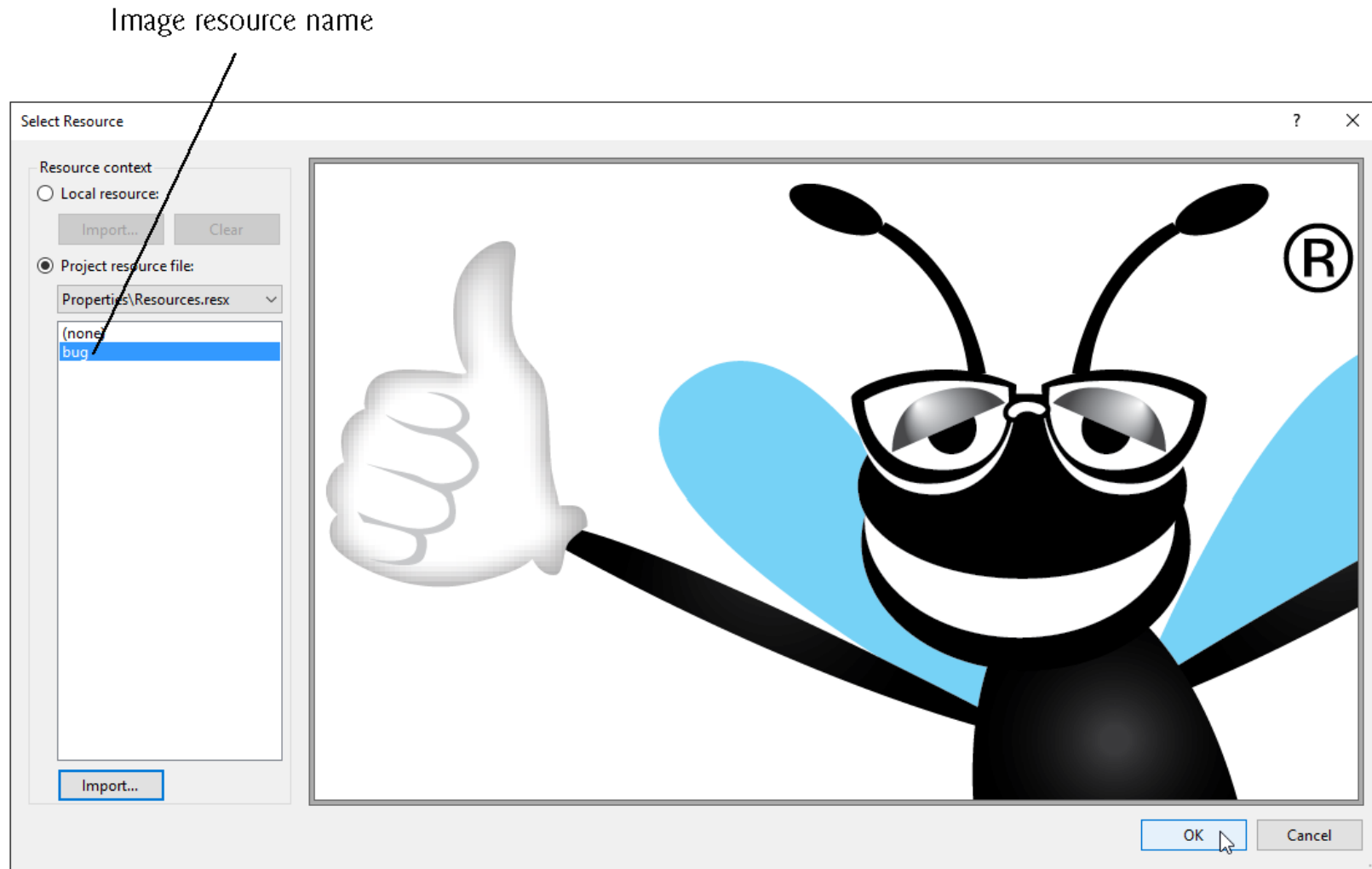


Fig. 2.32 | Select Resource dialog displaying a preview of selected image.

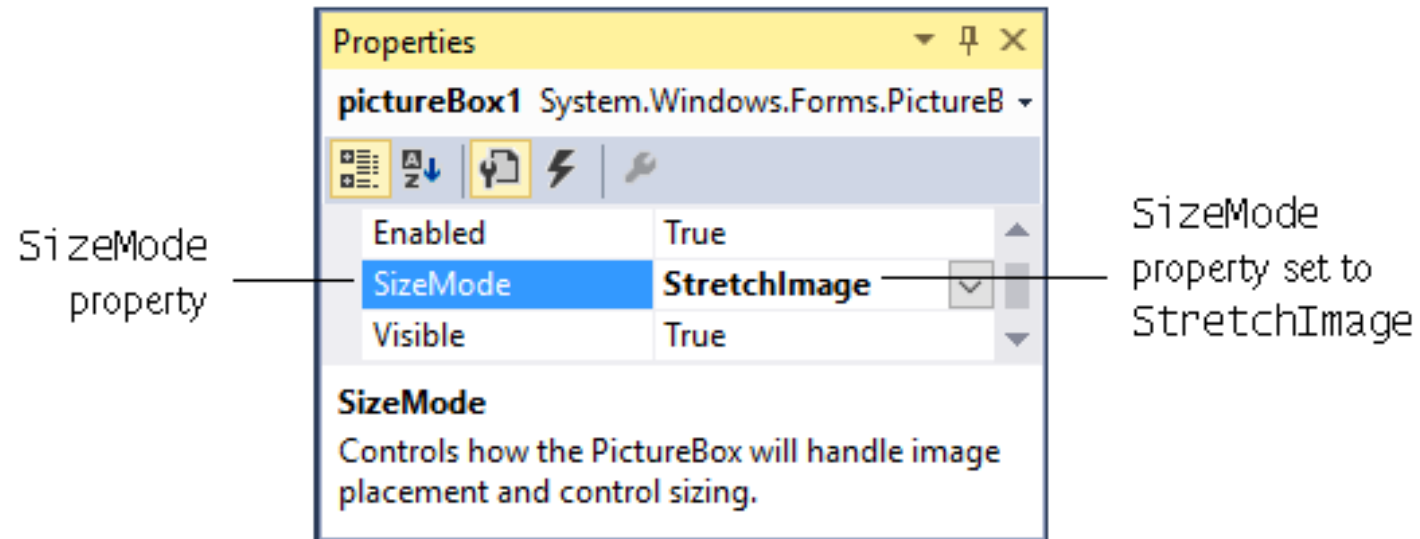


Fig. 2.33 | Scaling an image to the size of the **PictureBox**.

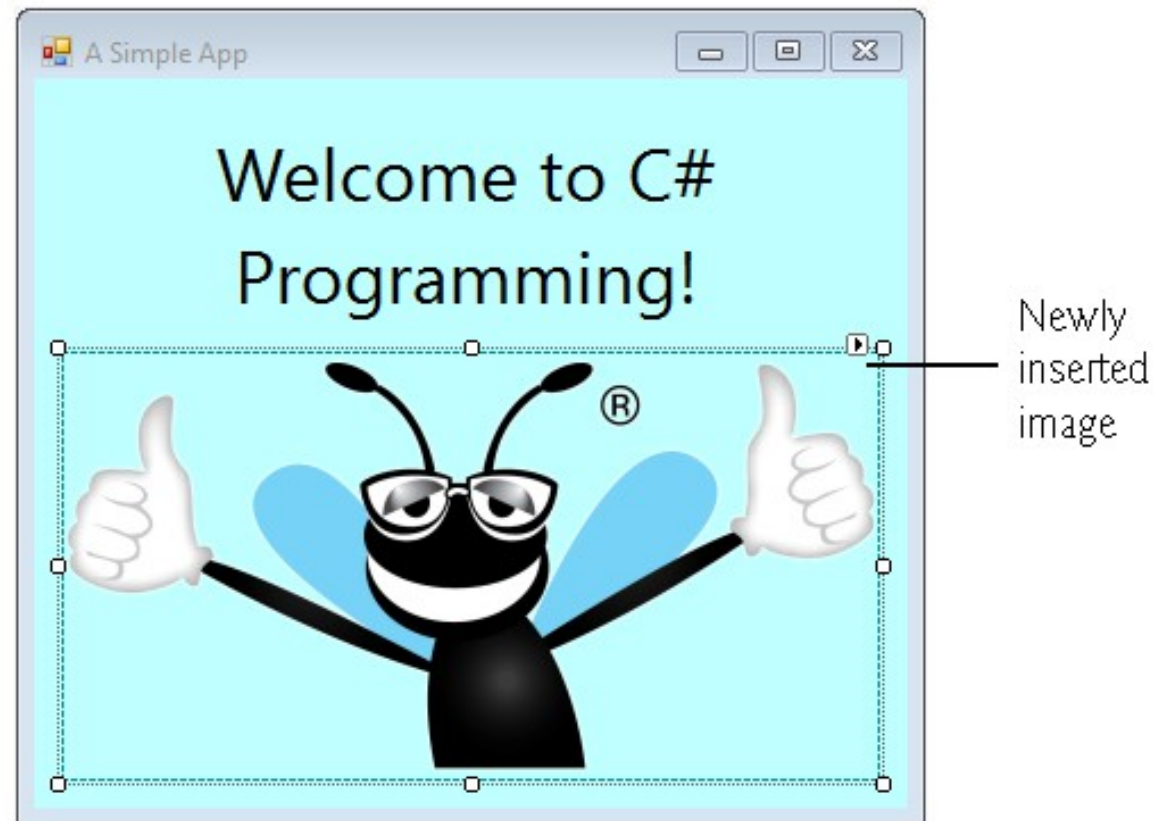


Fig. 2.34 | PictureBox displaying an image.

2.6 Visual Programming: Creating a Simple App that Displays Text and an Image



- ▶ Select **File > Save All** to save the entire solution.
- ▶ The solution file (which has the filename extension `.sln`) contains the name and location of its project, and the project file (which has the filename extension `.csproj`) contains the names and locations of all the files in the project.
- ▶ If you want to reopen your project at a later time, simply open its `.sln` file.

2.6 Visual Programming: Creating a Simple App that Displays Text and an Image



- ▶ In **run mode**, the app is executing, and you can interact with only a few IDE features—features that are not available are disabled (grayed out).
- ▶ Select **Debug > Start Debugging** to execute the app (or press the *F5* key).
- ▶ The IDE enters run mode and displays “(Running)” next to the app’s name in the IDE’s title bar.
- ▶ Click the app’s close box (X) to terminate the app

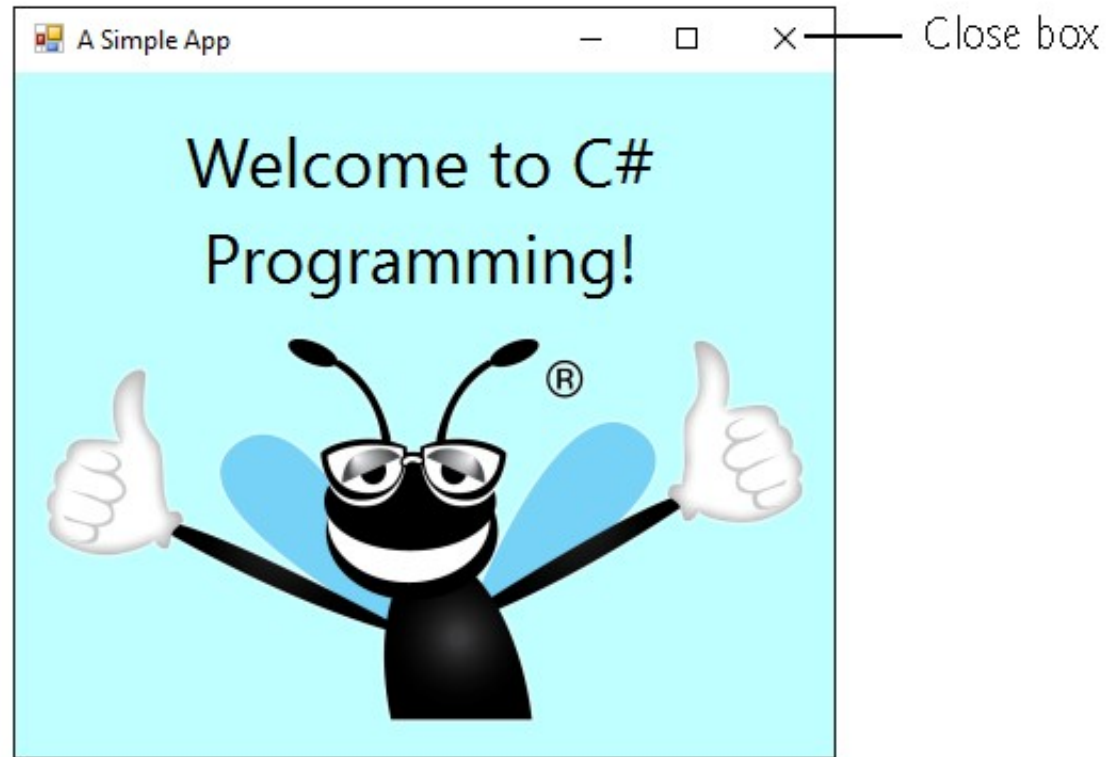


Fig. 2.35 | IDE in run mode, with the running app in the foreground.